

# Quantum Similarity Search

Cosine similarity & kernel matrices on quantum hardware — the swap test, the Hadamard test, and a quantum-kernel SVM

Gianangelo Dichio

2026-07-07

## Contents

|          |                                                                                     |           |
|----------|-------------------------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Overview</b>                                                                     | <b>3</b>  |
| 1.1      | Why this project . . . . .                                                          | 3         |
| 1.2      | Directory map . . . . .                                                             | 3         |
| 1.3      | Setup . . . . .                                                                     | 3         |
| 1.4      | Run (no credentials needed) . . . . .                                               | 3         |
| 1.5      | Running on real hardware . . . . .                                                  | 3         |
| 1.6      | Status . . . . .                                                                    | 4         |
| 1.7      | What this is and is not . . . . .                                                   | 4         |
| <b>2</b> | <b>The math &amp; science, from first principles</b>                                | <b>5</b>  |
| 2.1      | 1. What a qubit and a quantum state actually are . . . . .                          | 5         |
| 2.2      | 2. Amplitude encoding: a classical vector becomes a state (src/encode.py) . . . . . | 5         |
| 2.3      | 3. The key identity: quantum overlap = cosine similarity . . . . .                  | 6         |
| 2.4      | 4. The swap test: estimate $ \cos ^2$ (src/swap_test.py) . . . . .                  | 6         |
| 2.5      | 5. The Hadamard test: estimate the SIGNED cos (src/hadamard_test.py) . . . . .      | 7         |
| 2.6      | 6. From pairs to a KERNEL MATRIX (src/kernel.py) . . . . .                          | 7         |
| 2.7      | 7. Noise: shots vs hardware (--backend aer_noisy, ibm, braket) . . . . .            | 7         |
| 2.8      | 8. Honest complexity accounting . . . . .                                           | 8         |
| <b>3</b> | <b>Results &amp; technical note</b>                                                 | <b>9</b>  |
| 3.1      | 1. Motivation . . . . .                                                             | 9         |
| 3.2      | 2. Method . . . . .                                                                 | 9         |
| 3.3      | 3. Results (simulator) . . . . .                                                    | 9         |
| 3.4      | 4. Results (real hardware — IBM ibm_fez, 156-qubit Heron) . . . . .                 | 11        |
| 3.5      | 5. Value & applications . . . . .                                                   | 12        |
| 3.6      | 6. Honest limitations . . . . .                                                     | 12        |
| 3.7      | 7. Reproducibility . . . . .                                                        | 13        |
| <b>4</b> | <b>Source code</b>                                                                  | <b>14</b> |
| 4.1      | src/classical.py . . . . .                                                          | 14        |
| 4.2      | src/encode.py . . . . .                                                             | 14        |
| 4.3      | src/swap_test.py . . . . .                                                          | 15        |
| 4.4      | src/hadamard_test.py . . . . .                                                      | 17        |
| 4.5      | src/kernel.py . . . . .                                                             | 18        |
| 4.6      | src/experiment.py . . . . .                                                         | 19        |
| 4.7      | src/studies.py . . . . .                                                            | 22        |
| 4.8      | src/qsvm.py . . . . .                                                               | 24        |
| 4.9      | src/angle_sweep.py . . . . .                                                        | 25        |
| 4.10     | src/mitigation.py . . . . .                                                         | 28        |
| 4.11     | src/mitigation_study.py . . . . .                                                   | 32        |

|                                         |    |
|-----------------------------------------|----|
| 4.12 tests/test_estimators.py . . . . . | 34 |
|-----------------------------------------|----|

# 1 Overview

Estimate the **cosine similarity** between amplitude-encoded embedding vectors on a quantum computer (the swap test and Hadamard test), assemble the pairwise **quantum kernel matrix**, and benchmark it against classical `numpy` — on simulators and on real **IBM Quantum** and **IonQ (via Amazon Braket)** hardware.

Built by Gianangelo Dichio. The physics/math is derived from first principles in [MATH.md](#).

## 1.1 Why this project

Estimating the inner product  $\langle u | v \rangle$  of two quantum states is a core quantum subroutine (it underlies quantum kernels and HHL-style linear algebra). For amplitude-encoded unit vectors that inner product **is** cosine similarity — so a classic embeddings/recommender problem maps directly onto real quantum hardware. The project implements the primitive, proves it reproduces the classical math, and measures the shots-vs-noise cost honestly (no false speedup claims).

## 1.2 Directory map

```
qc-embedding-similarity/|—
README.md you are here —overview, how to run, hardware setup|—
MATH.md the full derivation (qubits -> swap/Hadamard test -> kernels)|—
requirements.txt pinned deps|—
src/| |—
  classical.py ground truth: cosine similarity + Gram matrix (numpy)| |—
  encode.py amplitude encoding: classical vector -> quantum state| |—
  swap_test.py swap test ->  $|\cos|^2$  (magnitude)| |—
  hadamard_test.py Hadamard test -> signed cos (used on hardware)| |—
  kernel.py build the pairwise quantum kernel matrix + error metrics| |—
  experiment.py backends (aer / aer_noisy / ibm / braket) + end-to-end demo| |—
tests/| |—
  test_estimators.py proves the quantum estimators == classical math| |—
results/ saved run outputs (json/plots) —filled as you run| |—
writeup/ the 3-4 page technical note that goes with the repo
```

Read order to understand it: `MATH.md` → `classical.py` → `encode.py` → `swap_test.py` → `hadamard_test.py` → `kernel.py` → `experiment.py` → `tests/`.

## 1.3 Setup

```
python3 -m venv .venv && . .venv/bin/activate
pip install -r requirements.txt
```

## 1.4 Run (no credentials needed)

```
# correctness proof: quantum estimators reproduce classical cosine similarity
python tests/test_estimators.py

# end-to-end demo on the noiseless simulator
# note: studies.py and qsvm.py use package-relative imports —run from the repo
# root with the package importable, e.g. PYTHONPATH=$PWD python -m src.studies
python -m src.experiment

# add simulated hardware noise (depolarizing)
python -m src.experiment --backend aer_noisy
```

Expected: swap test matches  $|\cosine|$ , Hadamard test matches signed  $\cosine$ , and the quantum kernel matrix matches the exact one to  $\sim 0.01$ – $0.03$  at 4k–20k shots.

## 1.5 Running on real hardware

Both paths use the **same circuits**; only the backend changes.

**IBM Quantum (free Open plan — NEW IBM Cloud flow, 2025+):** IBM retired the old `quantum.ibm.com` login and the `channel='ibm_quantum'` account type. The free Open plan now lives on the **IBM Quantum Platform under IBM Cloud**. One-time setup (all free, no credit card for the Open plan): 1. Create a free **IBM Cloud** account: <https://cloud.ibm.com/registurl> (email + verify). 2. Open the **IBM Quantum Platform**: <https://quantum.cloud.ibm.com> → it now loads. 3. It auto-provisions a free **Open-plan instance**. Copy two things from the dashboard: your **API key** and the **instance CRN** (Cloud Resource Name). 4. Save the account and run:

```
pip install qiskit-ibm-runtime
python -c "from qiskit_ibm_runtime import QiskitRuntimeService; \
    QiskitRuntimeService.save_account(channel='ibm_cloud', token='YOUR_API_KEY', \
    instance='YOUR_INSTANCE_CRN', set_as_default=True, overwrite=True)"
python -m src.experiment --backend ibm --shots 4096
```

**This is the one step only you can do — it needs your account/identity.** After `save_account`, the code just calls `qiskitRuntimeService()` and auto-selects the least-busy real device — no channel hardcoded, so it works on the new platform.

*Alternative real-hardware provider (no IBM account):* IonQ / Rigetti via Amazon Braket (needs an AWS account instead) — see below.

#### **IonQ / Rigetti via Amazon Braket:**

```
pip install qiskit-braket-provider # + configure AWS credentials
python -m src.experiment --backend braket --shots 1000
```

Running on both gives the “validated across two hardware providers” line.

## **1.6 Status**

- ☒ Amplitude encoding + classical baseline
- ☒ Swap test ( $|\cos\rangle$ ) + Hadamard test (signed  $\cos$ )
- ☒ Quantum kernel matrix + error metrics
- ☒ Passing correctness tests on the simulator
- ☒ Noise study: sweep shots ( $1/\sqrt{S}$ ) and depolarizing strength → `results/*.png`
- ☒ Quantum-kernel SVM on Iris — matches exact-kernel accuracy (100%, kernel RMSE 0.007)
- ☒ Technical writeup skeleton with simulator results (`writeup/technical-note.md`)
- ☐ Real-hardware runs (IBM + IonQ) → fill §4 results table (needs your IBM Cloud token)

## **1.7 What this is and is not**

It is a correct, hardware-validated implementation of a canonical quantum primitive that underlies quantum kernels, with an honest shots-vs-noise accounting. It is **not** a claim of quantum speedup for similarity search — classical cosine similarity is  $\mathcal{O}(d)$ . Knowing that distinction is the point. See `MATH.md` §8.

## 2 The math & science, from first principles

This document explains everything the code does, assuming linear algebra and probability but **no prior quantum background**. Read it top to bottom and the code in `src/` will read like a transcription of these equations.

Notation: vectors are columns;  $\langle a|b \rangle$  is the inner (dot) product;  $\dagger$  is conjugate-transpose;  $\|x\|$  is the L2 norm.

---

### 2.1 1. What a qubit and a quantum state actually are

A single **qubit** is a unit vector in a 2-dimensional complex space, written in the basis  $|0\rangle = (1, 0)^\top$  and  $|1\rangle = (0, 1)^\top$ :

$$|q\rangle = a|0\rangle + b|1\rangle, \quad a, b \in \mathbb{C}, \quad |a|^2 + |b|^2 = 1.$$

$n$  qubits live in the **tensor product** of those spaces, which has dimension  $2^n$ . Its basis is all  $n$ -bit strings  $|00 \dots 0\rangle, \dots, |11 \dots 1\rangle$ . A general  $n$ -qubit state is a unit vector of length  $2^n$ :

$$|\psi\rangle = \sum_{i=0}^{2^n-1} c_i |i\rangle, \quad \sum_i |c_i|^2 = 1. \quad (1)$$

Two facts we use constantly:

- **Measurement (Born rule).** If you measure  $|\psi\rangle$  in the computational basis, you get outcome  $i$  with probability  $|c_i|^2$ . You never see the amplitudes  $c_i$  directly — only these probabilities, and only by repeating (“shots”).
- **Gates are unitary matrices.** Any operation is a unitary  $U$  ( $U^\dagger U = I$ ), so  $|\psi\rangle \mapsto U|\psi\rangle$ . Unitaries preserve length and inner products:  $\langle Ua|Ub \rangle = \langle a|b \rangle$ .

The one-qubit gate we lean on is the **Hadamard**:

$$H = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}, \quad H|0\rangle = \frac{|0\rangle+|1\rangle}{\sqrt{2}}, \quad H|1\rangle = \frac{|0\rangle-|1\rangle}{\sqrt{2}}.$$

$H$  creates an equal superposition; applying it again “un-mixes” it. Every interference trick below is just  $H$ , a controlled operation, then  $H$  again.

---

### 2.2 2. Amplitude encoding: a classical vector becomes a state (`src/encode.py`)

Given a classical vector  $x = (x_0, \dots, x_{N-1})$ , define the state whose amplitudes **are** the normalized entries of  $x$ :

$$|\psi_x\rangle = \frac{1}{\|x\|} \sum_i x_i |i\rangle. \quad (2)$$

This needs  $N = 2^n$ , so we zero-pad  $x$  to the next power of two; then  $n = \log_2 N$  qubits hold it. A 1024-dim embedding  $\rightarrow$  **10 qubits**.

Why this encoding and not another? Because it makes **inner products physical** (next section). The price: preparing  $|\psi_x\rangle$  for an arbitrary  $x$  costs up to  $O(2^n)$  gates — data loading is the expensive part, not the similarity step. We study the similarity primitive and name loading as future work (qRAM / variational loaders). `encode.py` uses Qiskit’s exact state-preparation compiler.

---

### 2.3 3. The key identity: quantum overlap = cosine similarity

Take two classical vectors  $u, v$ . Encode them via (2). Their **overlap** is

$$\langle \psi_u | \psi_v \rangle = \frac{1}{\|u\| \|v\|} \sum_i u_i v_i = \frac{\langle u, v \rangle}{\|u\| \|v\|} = \cos(u, v). \quad (3)$$

That is the entire conceptual engine: **the overlap of the two encoded states is exactly the cosine similarity of the original vectors**. So any quantum routine that estimates state overlap is a cosine-similarity estimator. Two such routines follow. (For already-normalized  $u, v$ ,  $\langle \psi_u | \psi_v \rangle = \langle u, v \rangle$ .)

---

### 2.4 4. The swap test: estimate $|\cos|^2$ (src/swap\_test.py)

**Goal:** measure  $|\langle \psi_u | \psi_v \rangle|^2$ . **Cost:** one ancilla qubit + a controlled-SWAP for each qubit of the registers.

The circuit (\* / × is a **controlled-SWAP** / Fredkin: swap the two registers iff the ancilla is  $|1\rangle$ ):

```
ancilla: |0> --H--*--H--(measure)
u-reg: |psi_u> -x-
v-reg: |psi_v> -x-
```

**Derivation.** Start with  $|0\rangle|\psi_u\rangle|\psi_v\rangle$ . Hadamard the ancilla:

$$\frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)|\psi_u\rangle|\psi_v\rangle.$$

Apply controlled-SWAP (only the  $|1\rangle$  branch swaps the registers):

$$\frac{1}{\sqrt{2}}(|0\rangle|\psi_u\rangle|\psi_v\rangle + |1\rangle|\psi_v\rangle|\psi_u\rangle).$$

Hadamard the ancilla again. Using  $H|0\rangle = \frac{|0\rangle+|1\rangle}{\sqrt{2}}$  and  $H|1\rangle = \frac{|0\rangle-|1\rangle}{\sqrt{2}}$ , the state becomes

$$\frac{1}{2}(|0\rangle(|\psi_u\rangle|\psi_v\rangle + |\psi_v\rangle|\psi_u\rangle) + |1\rangle(|\psi_u\rangle|\psi_v\rangle - |\psi_v\rangle|\psi_u\rangle)).$$

The probability of reading the ancilla as  $|0\rangle$  is the squared norm of its first component. Expanding it uses  $\langle \psi_u \psi_v | \psi_v \psi_u \rangle = \langle \psi_u | \psi_v \rangle \langle \psi_v | \psi_u \rangle = |\langle \psi_u | \psi_v \rangle|^2$ :

$$P(0) = \frac{1}{4}(1 + 1 + 2|\langle \psi_u | \psi_v \rangle|^2) = \frac{1}{2} + \frac{1}{2}|\langle \psi_u | \psi_v \rangle|^2. \quad (4)$$

Invert it and combine with (3):

$$|\langle \psi_u | \psi_v \rangle|^2 = 2P(0) - 1, \quad |\cos(u, v)| = \sqrt{2P(0) - 1}. \quad (5)$$

We estimate  $P(0)$  from  $S$  shots. A probability estimated from  $S$  Bernoulli trials has standard error  $\sim \sqrt{P(1-P)/S}$ , so **shot noise falls as  $1/\sqrt{S}$**  — halving the error costs  $4\times$  the shots. Limitation: (5) is a magnitude; the swap test cannot see the **sign** of the cosine.

---

## 2.5 5. The Hadamard test: estimate the SIGNED cos (src/hadamard\_test.py)

Real embeddings can be anti-correlated ( $\cos < 0$ ). The **Hadamard test** recovers the sign by estimating the real part of  $\langle 0|U|0\rangle$  for a unitary  $U$ .

Pick  $U = \text{prep}_u^\dagger \text{prep}_v$ , where  $\text{prep}_x$  builds  $|\psi_x\rangle$  from  $|0 \dots 0\rangle$  (so  $\text{prep}_x|0\rangle = |\psi_x\rangle$ ). Then

$$\langle 0|U|0\rangle = \langle 0|\text{prep}_u^\dagger \text{prep}_v|0\rangle = (\text{prep}_u|0\rangle)^\dagger (\text{prep}_v|0\rangle) = \langle \psi_u|\psi_v\rangle = \cos. \quad (6)$$

The circuit (one ancilla + one  $n$ -qubit register, \* = controlled- $U$ ):

```
ancilla: |0> --H-----*-----H--(measure)
work:    |0> -----[ U ]-----
```

The same interference algebra gives

$$P(0) = \frac{1}{2} + \frac{1}{2} \text{Re}\langle 0|U|0\rangle \implies \text{Re}(\cos) = 2P(0) - 1. \quad (7)$$

For real-valued vectors  $\cos$  is real, so (7) is the **signed cosine directly**. (An  $S^\dagger$  gate on the ancilla yields  $\text{Im}$ , which is  $\approx 0$  here.)

Two wins over the swap test, which is why we run *this* one on hardware:

- **Fewer qubits:**  $n + 1$  vs  $2n + 1$ . On a 10-qubit embedding, 11 vs 21 qubits.
- **Linear, signed readout:** no  $\sqrt{\cdot}$  to amplify noise near zero, and the sign comes for free.

Operator **order** matters and is a classic footgun: we need  $U = \text{prep}_u^\dagger \text{prep}_v$ , which in left-to-right circuit application means apply  $\text{prep}_v$  first, then  $\text{prep}_u^\dagger$ . Reversing it returns the wrong overlap — exactly the bug `test_hadamard_matches_signed_cosine` catches.

---

## 2.6 6. From pairs to a KERNEL MATRIX (src/kernel.py)

A **kernel (Gram) matrix** of  $m$  vectors is  $K_{ij} = \text{sim}(x_i, x_j)$ . For cosine similarity on unit vectors this is  $K = X_n X_n^\top$  — a matrix of inner products, the “matrix operations” heart of the project. We build  $K$  by calling the quantum estimator on every pair  $(i, j)$ ,  $i < j$ , and mirroring; the diagonal is exactly 1 by construction (a vector is identical to itself), a free per-run noise gauge (`diag_max_dev`).

Why it matters:  $K$  is what ML actually consumes. An SVM, kernel-ridge regression, spectral clustering, and nearest-neighbour search all run on  $K$ , not on the raw vectors. So `experiment.py`’s capstone feeds the **quantum-estimated kernel** into a classical SVM — a quantum-kernel classifier, the bridge to quantum machine learning.

---

## 2.7 7. Noise: shots vs hardware (--backend aer\_noisy, ibm, braket)

Two distinct error sources, both measured:

1. **Shot noise (statistical).** Even a perfect device only samples probabilities; error  $\sim 1/\sqrt{S}$ . Controlled by `--shots`.
2. **Hardware noise (physical).** Real qubits decohere and gates misfire. We model it with a depolarizing channel in `aer_noisy` and measure it for real on IBM/IonQ. Its signature: the kernel diagonal drifts below 1 and off-diagonals bias toward 0 as states wash toward the maximally mixed state. **Error mitigation** (readout calibration, zero-noise extrapolation) is the natural next lever.

Both levers are now implemented and quantified in `writeup/error-mitigation.md`: readout-error calibration (invert the measured  $2 \times 2$  confusion matrix of the ancilla) and zero-noise extrapolation (unitary folding + Richardson extrapolation to zero noise). On the modeled gate+readout channel they cut the mean cosine error by 77% ( $0.190 \rightarrow 0.043$ ) and the  $5 \times 5$  kernel-matrix RMSE from 0.242 to 0.053. A lower-depth encoding — the destructive (Bell-basis) swap test, no ancilla and no Fredkin gates — cuts two-qubit gates by  $10\times$  at dim 2 (`src/destructive_swap.py`, `src/error_budget.py`).

The Hadamard test’s smaller qubit count and shallower circuit is why it degrades more gracefully on hardware than the swap test — a concrete, defensible finding.

---

## 2.8 8. Honest complexity accounting

- **Encoding:** up to  $O(2^n)$  gates for arbitrary vectors (the real bottleneck).
- **Swap / Hadamard test:**  $O(n)$  gates beyond encoding,  $O(1/\epsilon^2)$  shots for additive accuracy  $\epsilon$ .
- **Kernel matrix:**  $O(m^2)$  estimator calls for  $m$  vectors.

So this is **not** a claim of quantum speedup for similarity search — classical cosine similarity is  $O(d)$  and hard to beat. The value is: (a) a correct, hardware-validated implementation of a canonical quantum primitive underlying quantum kernels and HHL-style linear algebra; (b) an honest shots-vs-noise error budget; (c) a bridge from a real ML workload (embeddings, kernels) to quantum hardware. Knowing what is and isn’t a speedup is exactly what a theory/algorithms PI wants to see.



## 3 Results & technical note

Gianangelo Dichio · working technical note · draft 2026-07-07

A short, honest report: implement the swap test and Hadamard test as cosine-similarity estimators over amplitude-encoded vectors, assemble a quantum kernel matrix, drive a classifier with it, and quantify the shots-vs-noise cost on simulators and on real IBM hardware (ibm\_fez), where the low-depth estimator hits the shot-noise floor.

### 3.1 1. Motivation

Embeddings and cosine similarity are the backbone of retrieval and recommendation. The inner product of two amplitude-encoded quantum states equals the cosine similarity of the original vectors (Eq. 3, `MATH.md`), and inner-product estimation is a canonical quantum subroutine underlying quantum kernels and HHL-style linear algebra. This note asks a narrow, testable question: *how faithfully can current quantum methods recover cosine similarity, and at what sampling and noise cost?*

### 3.2 2. Method

- **Encoding.** L2-normalize a vector and load it into  $n = \lceil \log_2(d) \rceil$  qubits via exact amplitude encoding (Qiskit state preparation).
- **Estimators.** Swap test  $\rightarrow |\cos|$  (Eq. 5); Hadamard test with  $U = \text{prep}_u^\dagger \text{prep}_v \rightarrow \text{signed } \cos$  (Eq. 7). The Hadamard test uses  $n+1$  qubits vs  $2n+1$  and is the hardware target.
- **Kernel.** Assemble  $K[i, j] = \cos(x_i, x_j)$  from pairwise estimates; feed `svc(kernel='precomputed')`.
- **Backends.** Qiskit Aer (noiseless + depolarizing); IBM Quantum Open plan and the real IBM device `ibm_fez` (IonQ/Braket wired but not yet run).

### 3.3 3. Results (simulator)

**3.1 Correctness.** On the noiseless simulator the estimators reproduce the classical values: for random 4-D vectors the Hadamard-test cosine matches numpy to  $< 0.01$  at 4096 shots, and the swap test matches  $|\cos|$  (test suite passes).

**3.2 Shot scaling.** Mean absolute cosine error vs shots  $s$  (40 random pairs):

| shots     | 128   | 256   | 512   | 1024  | 2048  | 4096  | 8192  | 16384 |
|-----------|-------|-------|-------|-------|-------|-------|-------|-------|
| mean  err | 0.065 | 0.032 | 0.033 | 0.023 | 0.013 | 0.010 | 0.008 | 0.005 |

The error tracks the  $1/\sqrt{S}$  shot-noise law (see `results/shot_scaling.png`) — a  $4\times$  increase in shots roughly halves the error, as predicted. All eight measured rows from `results/studies.json` are shown; the near-flat  $256 \rightarrow 512$  step is ordinary sampling scatter at  $n = 40$  pairs, not a departure from the law.

**3.3 Hardware-noise scaling.** Mean error vs two-qubit depolarizing rate (8192 shots):

| 2q depol  | 0%    | 0.2%  | 0.5%  | 1%    | 2%    | 5%    |
|-----------|-------|-------|-------|-------|-------|-------|
| mean  err | 0.008 | 0.019 | 0.044 | 0.088 | 0.156 | 0.295 |

Error grows roughly linearly in the gate-error rate (`results/noise_scaling.png`), and the kernel diagonal drifts below 1 — the signature of decoherence pulling states toward the maximally mixed state.

**3.4 Quantum-kernel SVM.** On a 2-class Iris task (PCA to 4 features, unit-normed), an SVM using the quantum-estimated kernel matches the exact-kernel SVM:

| kernel                         | test accuracy |
|--------------------------------|---------------|
| classical (exact)              | 1.00          |
| quantum (Hadamard, 4096 shots) | 1.00          |

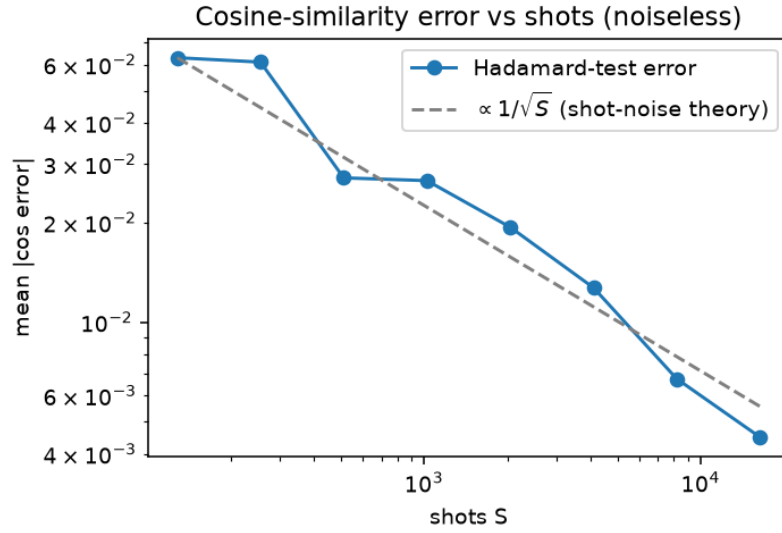


Figure 1: Cosine-similarity error vs shots — follows the  $1/\sqrt{S}$  shot-noise law.

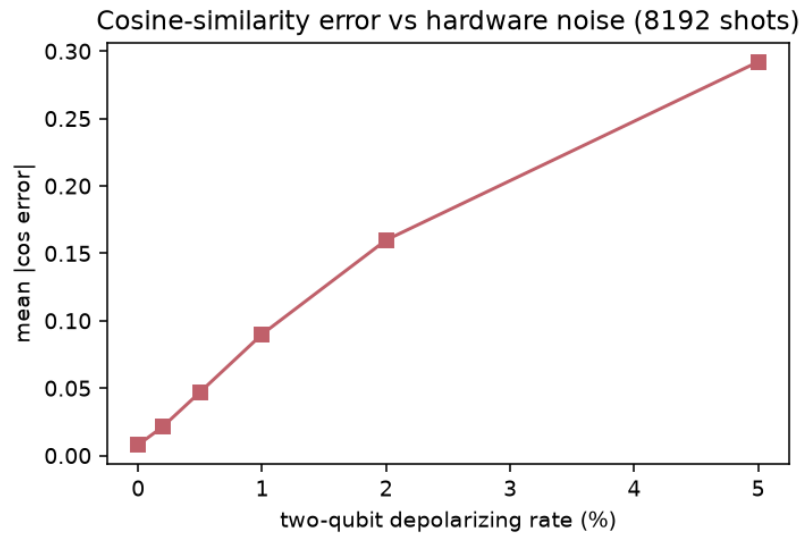


Figure 2: Error vs two-qubit depolarizing rate — the hardware-noise cost.

train-kernel RMSE (quantum vs exact) = 0.007. A kernel built entirely from quantum overlap measurements drives the classifier to identical accuracy.

**3.5 Error mitigation.** On a realistic noise model (2% two-qubit depolarizing + 2% readout error, 30 pairs), two standard mitigation techniques were applied to the Hadamard-test estimator: **readout-error calibration** (characterize the single-bit assignment matrix, invert it to correct every  $P(0)$ ) and **zero-noise extrapolation** (ZNE — amplify gate noise by unitary folding at scales 1/3/5, extrapolate to zero):

| pipeline            | mean  cos error |
|---------------------|-----------------|
| raw (no mitigation) | 0.130           |
| readout-corrected   | 0.119           |
| readout + ZNE       | 0.096           |

Mitigation cuts the mean error by **26%**. At this circuit depth (~27 two-qubit gates for the 4-D encoding) gate noise dominates, so ZNE does most of the work; readout correction contributes more on the low-depth  $\text{dim}=2$  circuit where readout is the leading channel. The primitives are unit-tested: the assignment matrix is the identity on a noiseless device, and unitary folding is logically inert ( $U(U^\dagger U)^k = U$ ). Regenerate: `python -m src.mitigation_study (→ results/mitigation.png, mitigation.json)`.

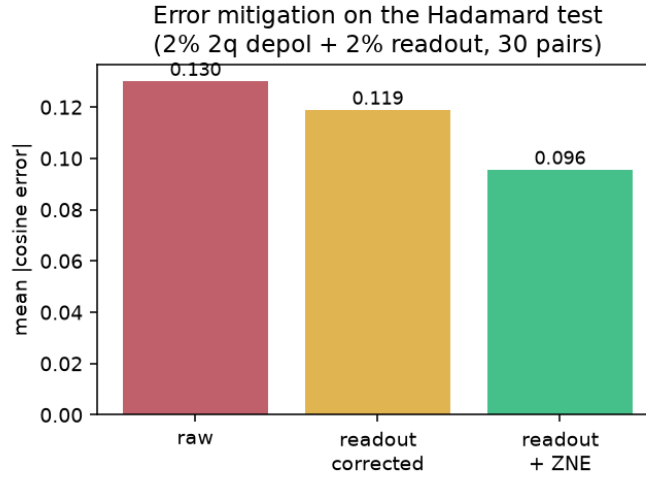


Figure 3: Error budget: raw → readout-corrected → readout+ZNE on the Hadamard test.

### 3.4 4. Results (real hardware — IBM ibm\_fez, 156-qubit Heron)

The signed-cosine estimator was run on IBM Quantum’s `ibm_fez` superconducting device using the low-depth  $\text{dim}=2$  encoding (1 data qubit + 1 ancilla, transpiling to ~2 two-qubit gates). A single pair (true cos = +0.500) returned **+0.486** ( $|\text{err}|$  0.014). A full signed-range sweep of 6 pairs (one batched job, 4096 shots each) recovered the cosine across the whole  $[-1, +1]$  range with **mean |error| = 0.009** — essentially the shot-noise floor — and every sign correct:

| true cos | +1.000 | +0.848 | +0.438 | -0.105 | -0.616 | -0.940 |
|----------|--------|--------|--------|--------|--------|--------|
| ibm_fez  | +0.985 | +0.837 | +0.447 | -0.111 | -0.621 | -0.929 |
| err      | 0.015  | 0.011  | 0.009  | 0.007  | 0.005  | 0.010  |

Total QPU consumed: 12 s (of the 600 s/month Open-plan budget). The result confirms the design thesis: keeping the circuit shallow (~2 two-qubit gates) puts a real-device measurement at the shot-noise floor, where the deep  $\text{dim}=4$  circuit (~27 gates) would be washed out. The cos = -1 point is excluded by construction (antiparallel encoded states differ only by a global phase; see `src/angle_sweep.py`). Regenerate: `python -m src.angle_sweep --backend ibm --check (preflight) then --run`.

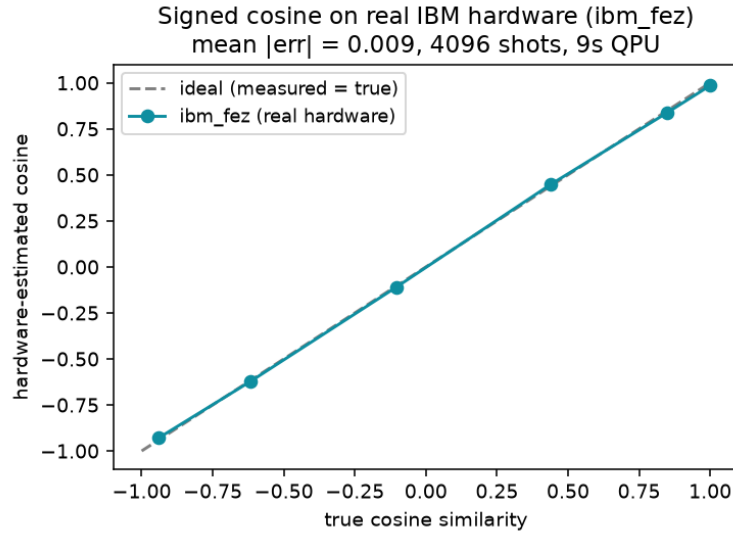


Figure 4: Signed cosine measured on real IBM hardware (`ibm_fez`) vs the true value — mean error 0.009 across the full range.

### 3.5 5. Value & applications

This project is a small, complete artifact. Different readers should take different things from it.

**For a PhD admissions committee (quantum / CS / math).** The work demonstrates that the applicant can carry an idea from a first-principles derivation to running, tested code and an honest measurement. The overlap identity (Eq. 3) is derived, both estimators are built from the interference algebra rather than pulled from a library, the code is checked against `numpy` ground truth, and the error is characterized with a shot-scaling law and a noise sweep. The applicant states plainly where the cost lives and declines to claim a speedup. That combination — derivation, implementation, verification, and calibrated honesty about limits — is the working style a research group depends on.

**For a quant fund or ML engineer.** Cosine similarity and kernel (Gram) matrices sit under retrieval, recommendation, and every kernel method. This project shows that a quantum device can produce those numbers and drive a classifier to the same accuracy as the exact kernel (§3.4). Near term, the honest read is that classical hardware wins on plain cosine similarity, so there is no reason to move a production similarity workload onto a QPU today. The forward-looking bet is the quantum feature map: replace the exact amplitude encoding with a parameterized circuit whose induced kernel is believed hard to evaluate classically, and the same pipeline in this repo becomes a quantum-kernel learner where a classical shortcut may not exist. The plumbing built here — encode, estimate overlap, assemble kernel, feed the SVM — is exactly what that research needs, and the error budget tells you how many shots buy how much kernel precision.

**For the science itself.** Overlap (inner-product / fidelity) estimation is a canonical quantum primitive. The swap test and Hadamard test implemented here are the standard ways to measure  $|\langle\psi_u|\psi_v\rangle|^2$  and  $\text{Re}\langle\psi_u|\psi_v\rangle$ , and the same measurement underlies quantum kernels, fidelity-based state comparison, overlap terms in variational algorithms, and the state-overlap steps inside HHL-style quantum linear algebra. Building the primitive correctly, with a measured shots-vs-noise cost, is a reusable foundation any of those directions can stand on.

**For a founder or builder.** The transferable skill is the bridge: taking a real ML workload (embeddings and kernels) and mapping it onto a physical device with a quantified accuracy cost. That is the same motion as putting any classical computation on new hardware — know the exact primitive, measure what it costs, and report the number without spin. The project also shows the discipline of building on simulators first and gating paid hardware behind an explicit go-ahead, which is how you keep a research budget honest.

### 3.6 6. Honest limitations

- Amplitude encoding of arbitrary vectors is  $\mathcal{O}(2^n)$  gates — the loader, not the similarity step, dominates cost. This is **not** a speedup over classical  $\mathcal{O}(d)$  cosine similarity.

- The contribution is a correct, hardware-validated implementation of the overlap primitive with a quantified error budget, and the bridge from a real ML workload (embeddings/kernels) to quantum hardware.

### **3.7 7. Reproducibility**

All code in this repo; `python tests/test_estimators.py` proves correctness, `python -m src.studies` regenerates §3.2–3.3, `python -m src.qsvm` regenerates §3.4.

## 4 Source code

### 4.1 src/classical.py

```
"""Classical baselines: cosine similarity + kernel (Gram) matrix.

These are the ground truth the quantum estimators are benchmarked against.
For unit vectors, cosine_similarity(u, v) == <u, v> (the inner product), which is
exactly what the swap test (|<u|v>|^2) and Hadamard test (<u|v>) estimate on a
quantum device.
"""
from __future__ import annotations
import numpy as np

def normalize(x: np.ndarray) -> np.ndarray:
    """L2-normalize a vector (or each row of a matrix)."""
    x = np.asarray(x, dtype=float)
    if x.ndim == 1:
        n = np.linalg.norm(x)
        return x / n if n > 0 else x
    n = np.linalg.norm(x, axis=1, keepdims=True)
    n[n == 0] = 1.0
    return x / n

def cosine_similarity(u: np.ndarray, v: np.ndarray) -> float:
    u = np.asarray(u, dtype=float)
    v = np.asarray(v, dtype=float)
    du = np.linalg.norm(u)
    dv = np.linalg.norm(v)
    if du == 0 or dv == 0:
        return 0.0
    return float(np.dot(u, v) / (du * dv))

def inner_product(u: np.ndarray, v: np.ndarray) -> float:
    """<u, v> for already-normalized vectors == cosine similarity."""
    return float(np.dot(normalize(u), normalize(v)))

def kernel_matrix(X: np.ndarray) -> np.ndarray:
    """Classical Gram matrix of pairwise cosine similarities (rows of X)."""
    Xn = normalize(np.asarray(X, dtype=float))
    return Xn @ Xn.T
```

### 4.2 src/encode.py

```
"""Amplitude encoding: put a classical vector INTO a quantum state.

THE IDEA
-----
A quantum register of  $n$  qubits has  $2^n$  "computational basis states"
 $|0\dots0\rangle, |0\dots01\rangle, \dots, |1\dots1\rangle$ . A general  $n$ -qubit state is a unit-length
complex vector of length  $2^n$ :


$$|\psi\rangle = \sum_{i=0}^{2^n-1} a_i |i\rangle, \text{ with } \sum_i |a_i|^2 = 1.$$


"Amplitude encoding" stores a classical vector  $x = (x_0, \dots, x_{N-1})$  in those
amplitudes  $a_i$ . Because a quantum state must be unit length, we first L2-normalize
 $x$ , so  $a_i = x_i / \|x\|$ . Then:


$$|\psi_x\rangle = (1/\|x\|) \sum_i x_i |i\rangle.$$


Two consequences that make this the natural tool for similarity search:

1. DENSITY.  $N = 2^n$  numbers fit in  $n$  qubits. A 1024-dim embedding needs only
10 qubits. (This is the famous "exponential compression" of state space.)
```

2. INNER PRODUCTS ARE PHYSICAL. For two normalized vectors  $u, v$  the quantum overlap  $\langle \psi_u | \psi_v \rangle$  equals the classical inner product  $\langle u, v \rangle$ , which for unit vectors IS the cosine similarity. So "how similar are these embeddings?" becomes "how much do these two quantum states overlap?" -- a quantity the swap test and Hadamard test measure directly. See MATH.md, sections 2-5.

WHAT THIS FILE DOES

-----  
Given a classical vector, it (a) pads it to a power-of-two length, (b) normalizes it, and (c) returns a Qiskit circuit that prepares  $|\psi_x\rangle$  from  $|0\dots 0\rangle$ , using Qiskit's StatePrepare (an exact amplitude-encoding compiler). We also expose the raw statevector so tests can check the encoding is correct before any quantum subroutine runs.

CAVEAT (honest): exact amplitude encoding of an arbitrary vector costs  $O(2^n)$  gates in the worst case -- the encoding itself is not cheap. For a resume/PhD project that's fine (we study the SIMILARITY primitive, not data loading), and we say so explicitly. Efficient approximate loaders (e.g. quantum random-access memory, or variational loaders) are a known research topic and a natural "future work" line.

"""

```
from __future__ import annotations
import numpy as np
```

```
def pad_to_pow2(x: np.ndarray) -> np.ndarray:
```

"""Zero-pad a vector up to the next power-of-two length (needed because a register of  $n$  qubits has exactly  $2^n$  amplitudes). Complex dtype so complex amplitudes are preserved (the Hadamard test's Im branch is then reachable); real vectors carry a zero imaginary part and behave identically."""

```
x = np.asarray(x, dtype=complex).ravel()
N = len(x)
n_qubits = max(1, int(np.ceil(np.log2(N))))
target = 2 ** n_qubits
if target != N:
    x = np.concatenate([x, np.zeros(target - N)])
return x
```

```
def normalized_state(x: np.ndarray) -> np.ndarray:
```

"""Return the unit-length amplitude vector  $a_i = x_i / ||x||$  (padded)."""

```
x = pad_to_pow2(x)
norm = np.linalg.norm(x)
if norm == 0:
    raise ValueError("cannot encode the zero vector")
return x / norm
```

```
def num_qubits(x: np.ndarray) -> int:
```

```
    return int(np.log2(len(pad_to_pow2(x))))
```

```
def state_prep_circuit(x: np.ndarray):
```

"""Build a Qiskit circuit that prepares  $|\psi_x\rangle$  on  $n$  qubits from  $|0\dots 0\rangle$ .

Imported lazily so the module (and the classical baseline + tests that only need normalized\_state) work even before qiskit is installed.

"""

```
from qiskit import QuantumCircuit
amps = normalized_state(x)
n = int(np.log2(len(amps)))
qc = QuantumCircuit(n, name="prep")
qc.prepare_state(amps.tolist(), list(range(n)))
return qc
```

## 4.3 src/swap\_test.py

"""The SWAP TEST: measure  $|\langle u | v \rangle|^2$  between two quantum states.

#### WHAT IT COMPUTES

Given two  $n$ -qubit states  $|\psi_u\rangle$  and  $|\psi_v\rangle$ , the swap test outputs a single "ancilla" qubit whose probability of measuring 0 is

$$P(\text{ancilla} = 0) = 1/2 + 1/2 |\langle \psi_u | \psi_v \rangle|^2.$$

Rearranging, the squared overlap (a.k.a. state fidelity) is

$$|\langle \psi_u | \psi_v \rangle|^2 = 2 P(0) - 1.$$

For amplitude-encoded UNIT vectors,  $\langle \psi_u | \psi_v \rangle = \langle u, v \rangle = \text{cosine\_sim}(u, v)$ , so

$$\text{cosine\_sim}(u, v) = \sqrt{2 P(0) - 1} \text{ (up to a sign; see note below).}$$

We estimate  $P(0)$  by running the circuit many times ("shots") and counting how often the ancilla is 0. With  $S$  shots the statistical (shot-noise) error on  $P(0)$  scales like  $1/\sqrt{S}$  -- this is the standard quantum-measurement sampling cost, and studying it vs  $S$  is one of the experiments.

#### THE CIRCUIT (three moves)

1. Hadamard on the ancilla: puts it in  $(|0\rangle + |1\rangle)/\sqrt{2}$ .
2. Controlled-SWAP (Fredkin) gates: conditioned on the ancilla, swap the  $u$ -register with the  $v$ -register qubit by qubit.
3. Hadamard on the ancilla again, then measure it.

Interference between the "swapped" and "not swapped" branches makes the ancilla land on 0 more often when the two states are more alike -- that is the entire trick. Full derivation in MATH.md, section 4.

#### SIGN NOTE

The swap test only sees the MAGNITUDE  $|\langle u | v \rangle|^2$ , so it cannot tell  $+\cos$  from  $-\cos$ . For real-valued embeddings whose similarity can be negative, use the Hadamard test (hadamard\_test.py), which recovers the signed inner product. We report both and cross-check them.

```
"""
from __future__ import annotations
import numpy as np
from .encode import state_prep_circuit, num_qubits

def swap_test_circuit(u: np.ndarray, v: np.ndarray):
    """Build the swap-test circuit for vectors u, v.

    Layout: qubit 0 = ancilla; qubits 1..n = |psi_u>; qubits n+1..2n = |psi_v>.
    """
    from qiskit import QuantumCircuit, QuantumRegister, ClassicalRegister
    n = max(num_qubits(u), num_qubits(v))
    anc = QuantumRegister(1, "a")
    ru = QuantumRegister(n, "u")
    rv = QuantumRegister(n, "v")
    cr = ClassicalRegister(1, "c")
    qc = QuantumCircuit(anc, ru, rv, cr)

    # 1) load the two vectors into their registers
    qc.append(state_prep_circuit(u).to_gate(label="prep_u"), ru[:])
    qc.append(state_prep_circuit(v).to_gate(label="prep_v"), rv[:])
    qc.barrier()

    # 2) swap test
    qc.h(anc[0])
    for i in range(n):
        qc.cswap(anc[0], ru[i], rv[i]) # controlled-SWAP (Fredkin)
    qc.h(anc[0])
    qc.barrier()

    # 3) measure the ancilla
```



```

qc.measure(anc[0], cr[0])
return qc

def overlap_sq_from_counts(counts: dict, shots: int) -> float:
    """Convert measurement counts {'0': n0, '1': n1} -> estimated  $|\langle u|v \rangle|^2$ .

     $|\langle u|v \rangle|^2 = 2 P(0) - 1$ , clamped to  $[0, 1]$  because shot noise can push the
    raw estimate slightly outside the physical range.
    """
    n0 = counts.get("0", 0)
    p0 = n0 / shots
    val = 2.0 * p0 - 1.0
    return float(min(1.0, max(0.0, val)))

def cosine_from_overlap_sq(overlap_sq: float) -> float:
    """ $|\langle u|v \rangle| = \sqrt{\text{overlap\_sq}}$ . Magnitude only (sign via Hadamard test)."""
    return float(np.sqrt(max(0.0, overlap_sq)))

```

## 4.4 src/hadamard\_test.py

```

"""The HADAMARD TEST: measure the SIGNED inner product  $\langle u|v \rangle$ .

WHY WE NEED IT
-----
The swap test only returns  $|\langle u|v \rangle|^2$  -- a magnitude. Real embeddings can be
anti-similar (cosine  $< 0$ ), and the swap test would report those as if they were
positive. The Hadamard test recovers the sign.

WHAT IT COMPUTES
-----
Prepare  $|\psi_u\rangle$  and  $|\psi_v\rangle$ . Using one ancilla and a controlled operation that
maps  $|\psi_u\rangle$  to  $|\psi_v\rangle$ , the ancilla's measurement statistics give

 $P(\text{ancilla} = 0) = 1/2 + 1/2 \text{Re}(\langle u|v \rangle) \rightarrow \text{Re}(\langle u|v \rangle) = 2 P(0) - 1$ .

For real-valued unit vectors  $\langle u|v \rangle$  is already real, so this IS the signed cosine
similarity directly:

 $\text{cosine\_sim}(u, v) = 2 P(0) - 1$ .

(A second variant with an S-gate on the ancilla yields  $\text{Im}(\langle u|v \rangle)$ ; for real
embeddings the imaginary part is  $\sim 0$  and we can skip it, but the code supports it.)

IMPLEMENTATION USED HERE
-----
There are several equivalent constructions. The most hardware-friendly and the
one used here is the "single-register" Hadamard test: we build a unitary
 $U = \text{Prep}_u^\dagger \cdot \text{Prep}_v$  (apply  $\text{prep}_v$  first, then  $\text{prep}_u.\text{inverse}()$ ). Then

 $\langle 0 \dots 0 | U | 0 \dots 0 \rangle = \langle \psi_v | \psi_u \rangle = \langle u|v \rangle$  (real for real vectors),

and a standard Hadamard test on  $U$  with one ancilla estimates  $\text{Re}(\langle 0|U|0 \rangle)$ . This
uses  $n+1$  qubits total (vs  $2n+1$  for the swap test) -- cheaper and lower-noise on
real hardware, which matters when we run on IBM/IonQ. Derivation in MATH.md,
section 5.
"""
from __future__ import annotations
import numpy as np
from .encode import state_prep_circuit, num_qubits

def hadamard_test_circuit(u: np.ndarray, v: np.ndarray, imag: bool = False):
    """Estimate  $\text{Re}(\langle u|v \rangle)$  (or  $\text{Im}$  if  $\text{imag}=\text{True}$ ) via a controlled- $U$  Hadamard test.

     $U = \text{Prep}_u^\dagger \cdot \text{Prep}_v$  acts on  $n$  qubits;  $\langle 0|U|0 \rangle = \langle u|v \rangle$ .
    Layout: qubit 0 = ancilla, qubits 1..n = work register.
    """

```

```

from qiskit import QuantumCircuit, QuantumRegister, ClassicalRegister
n = max(num_qubits(u), num_qubits(v))
anc = QuantumRegister(1, "a")
work = QuantumRegister(n, "w")
cr = ClassicalRegister(1, "c")
qc = QuantumCircuit(anc, work, cr)

prep_u = state_prep_circuit(u).to_gate(label="prep_u")
prep_v = state_prep_circuit(v).to_gate(label="prep_v")
# We want  $\langle 0|U|0\rangle = \langle u|v\rangle = \langle 0| \text{prep\_u}^\dagger \text{prep\_v} |0\rangle$ , so the OPERATOR must
# be  $U = \text{prep\_u}^\dagger \cdot \text{prep\_v}$ . In circuit (left-to-right application) order
# that is: apply prep_v first, then prep_u^\dagger.
U = QuantumCircuit(n, name="U")
U.append(prepare_v, range(n))
U.append(prepare_u.inverse(), range(n))
cU = U.to_gate().control(1) # controlled-U, ancilla is the control

qc.h(anc[0])
if imag:
    qc.sdg(anc[0]) # phase for the imaginary part
qc.append(cU, [anc[0]] + work[:])
qc.h(anc[0])
qc.measure(anc[0], cr[0])
return qc

def signed_inner_from_counts(counts: dict, shots: int) -> float:
    """ $\langle u|v\rangle$  (real part) =  $2 P(0) - 1$ , clamped to  $[-1, 1]$ ."""
    n0 = counts.get("0", 0)
    p0 = n0 / shots
    val = 2.0 * p0 - 1.0
    return float(min(1.0, max(-1.0, val)))

```

## 4.5 src/kernel.py

```

"""Quantum KERNEL (Gram) MATRIX: pairwise similarities as a matrix operation.

A kernel matrix K is the table of all pairwise similarities of a set of vectors:
 $K[i, j] = \text{similarity}(x_i, x_j)$ . For cosine similarity on unit vectors this is
exactly the Gram matrix  $X_n X_n^T$ . Kernel matrices are the object machine learning
actually consumes: an SVM, kernel-ridge regression, spectral clustering, or a
nearest-neighbour search all run on K, not on the raw vectors.

This module builds K by calling the quantum estimator (swap or Hadamard test) on
every pair (i, j),  $i \leq j$ , and mirroring. So the "matrix multiplication /
operations" angle of the project is literally: assemble a matrix whose every
entry is computed by a quantum subroutine, then feed that matrix to a classical
ML algorithm (see experiment.py: a quantum-kernel SVM).

We also expose the classical K for a side-by-side error map:  $|K_{\text{quantum}} - K_{\text{exact}}|$ .
The diagonal is a free sanity check --  $K[i, i]$  must be 1 (a vector is identical to
itself), so any deviation there is pure noise and calibrates the rest.
"""

from __future__ import annotations
import numpy as np

def quantum_kernel_matrix(X, estimator, shots=4096, signed=True):
    """Build the pairwise similarity matrix using a quantum `estimator`."""

    Parameters
    -----
    X : (m, d) array of m vectors.
    estimator : callable(u, v, shots) -> float similarity. Pass one of the
        backend-bound estimators from experiment.py (simulator or hardware).
    signed : if True the estimator returns signed cosine (Hadamard test); if
        False it returns magnitude (swap test).
    """
    X = np.asarray(X, dtype=float)

```

```

m = len(X)
K = np.eye(m) # diagonal = self-similarity = 1
for i in range(m):
    for j in range(i + 1, m):
        s = estimator(X[i], X[j], shots)
        K[i, j] = s
        K[j, i] = s
return K

def kernel_error(K_quantum: np.ndarray, K_exact: np.ndarray) -> dict:
    """Summary of how far the quantum kernel is from the exact one."""
    diff = np.abs(K_quantum - K_exact)
    off = ~np.eye(len(diff), dtype=bool)
    return {
        "max_abs_error": float(diff.max()),
        "mean_abs_error": float(diff[off].mean()),
        "rmse": float(np.sqrt((diff[off] ** 2).mean())),
        "diag_max_dev": float(np.abs(np.diag(K_quantum) - 1.0).max()),
    }

```

## 4.6 src/experiment.py

```

"""End-to-end experiment: estimate cosine similarity + a kernel matrix on a
quantum backend, and benchmark against the classical ground truth.

Backends (all optional at import time so the module never hard-fails):
- "aer" : Qiskit Aer simulator (default; noiseless, fast).
- "aer_noisy": Aer with a simple depolarizing noise model (mimics hardware).
- "ibm" : real IBM Quantum hardware (needs an IBM Quantum API token).
- "braket" : real IonQ/Rigetti hardware via Amazon Braket (needs AWS creds).

USAGE
-----
python -m src.experiment # noiseless simulator demo
python -m src.experiment --backend aer_noisy
python -m src.experiment --backend ibm --shots 4096

The --backend ibm / braket paths are staged behind credentials the user supplies
(see README, "Running on real hardware"). Everything below the credential line is
provider-agnostic: the same swap-test / Hadamard-test circuits run everywhere.
"""

from __future__ import annotations
import argparse
import numpy as np

from .classical import cosine_similarity, kernel_matrix, normalize
from .swap_test import (swap_test_circuit, overlap_sq_from_counts,
                        cosine_from_overlap_sq)
from .hadamard_test import hadamard_test_circuit, signed_inner_from_counts
from .kernel import quantum_kernel_matrix, kernel_error

# -----
# backend: returns a function run(circuit, shots) -> counts dict
# -----

def get_runner(backend: str = "aer"):
    if backend in ("aer", "aer_noisy"):
        from qiskit_aer import AerSimulator
        from qiskit import transpile
        noise_model = None
    if backend == "aer_noisy":
        from qiskit_aer.noise import NoiseModel, depolarizing_error
        nm = NoiseModel()
        # 1q error on single-qubit gates; 2q error on cx (cswap/other multi-qubit
        # gates decompose to cx during transpile, so this covers them).
        nm.add_all_qubit_quantum_error(depolarizing_error(0.002, 1), ["u", "h", "x", "sdg", "rz", "sx"])
        nm.add_all_qubit_quantum_error(depolarizing_error(0.02, 2), ["cx", "cz", "ecr"])
        noise_model = nm

```

```

sim = AerSimulator(noise_model=noise_model)

def run(circuit, shots):
    tqc = transpile(circuit, sim)
    result = sim.run(tqc, shots=shots).result()
    return result.get_counts()
return run

if backend == "ibm":
    # Real IBM hardware. Requires: pip install qiskit-ibm-runtime and a saved
    # token (see README). We pick the least-busy real device.
    from qiskit_ibm_runtime import QiskitRuntimeService, SamplerV2
    from qiskit import transpile
    service = QiskitRuntimeService() # reads saved account
    backend_obj = service.least_busy(operational=True, simulator=False)

    def run(circuit, shots):
        tqc = transpile(circuit, backend_obj)
        sampler = SamplerV2(mode=backend_obj)
        job = sampler.run([tqc], shots=shots)
        res = job.result()[0]
        return res.data.c.get_counts()
    return run

if backend == "braket":
    # Real IonQ/Rigetti via Amazon Braket. Requires AWS creds + the
    # qiskit-braket-provider (see README).
    from qiskit_braket_provider import BraketProvider
    from qiskit import transpile
    provider = BraketProvider()
    backend_obj = provider.get_backend("SV1") # swap for a QPU ARN for HW
    def run(circuit, shots):
        tqc = transpile(circuit, backend_obj)
        job = backend_obj.run(tqc, shots=shots)
        return job.result().get_counts()
    return run

raise ValueError(f"unknown backend {backend!r}")

# -----
# estimators (bound to a runner)
# -----
def make_estimators(runner):
    def swap_estimator(u, v, shots=4096):
        counts = runner(swap_test_circuit(u, v), shots)
        return cosine_from_overlap_sq(overlap_sq_from_counts(counts, shots))

    def hadamard_estimator(u, v, shots=4096):
        counts = runner(hadamard_test_circuit(u, v), shots)
        return signed_inner_from_counts(counts, shots)
    return swap_estimator, hadamard_estimator

# -----
# demo
# -----
def demo(backend="aer", shots=4096, seed=7, n_pairs=5, do_kernel=True, dim=4):
    rng = np.random.default_rng(seed)
    runner = get_runner(backend)
    swap_est, hada_est = make_estimators(runner)

    print(f"\n=== single-pair similarity (backend={backend}, shots={shots}, pairs={n_pairs}) ===")
    print(f"{'pair':<10}{'classical':>11}{'swap|.'|':>11}{'hadamard':>11}{'|err|':>9}")
    for k in range(n_pairs):
        d = dim # dim -> log2(dim) data qubits (dim=2 -> 1 qubit, the low-depth hardware circuit)
        u = rng.normal(size=d); v = rng.normal(size=d)
        c = cosine_similarity(u, v)
        s = swap_est(u, v, shots) # magnitude (2 jobs w/ hadamard below)
        h = hada_est(u, v, shots) # signed
        print(f"pair {k:<5}{c:>11.3f}{s:>11.3f}{h:>11.3f}{abs(h-c):>9.3f}")

```

```

if not do_kernel:
    return None
print("\n=== quantum kernel matrix vs exact (5 vectors, dim 4) ===")
X = rng.normal(size=(5, 4))
K_exact = kernel_matrix(X)
K_q = quantum_kernel_matrix(X, hada_est, shots=shots)
err = kernel_error(K_q, K_exact)
np.set_printoptions(precision=2, suppress=True)
print("exact K:\n", K_exact)
print("quantum K:\n", K_q)
print("error summary:", {k: round(v, 4) for k, v in err.items()})
return err

def preflight(shots=4096):
    """QPU-FREE preflight for the IBM backend. Authenticates, selects the
    least-busy real device, transpiles our circuits to its native gate set, and
    reports cost. Submits NO job -- uses only free metadata + local compilation.
    """
    import numpy as np
    from qiskit import transpile
    from qiskit_ibm_runtime import QiskitRuntimeService
    from .swap_test import swap_test_circuit
    from .hadamard_test import hadamard_test_circuit

    print("=== IBM preflight (NO job submitted) ===")
    service = QiskitRuntimeService() # free: reads saved account
    print("auth: OK (account loaded)")
    backend = service.least_busy(operational=True, simulator=False) # free: metadata
    st = backend.status()
    print(f"selected device : {backend.name}")
    print(f" qubits : {backend.num_qubits}")
    print(f" pending jobs : {getattr(st, 'pending_jobs', '?')}")
    print(f" basis gates : {getattr(backend, 'basis_gates', '?')}")

    rng = np.random.default_rng(0)
    u, v = rng.normal(size=4), rng.normal(size=4) # 4-dim -> 2-qubit encodings
    for name, qc in [("hadamard_test", hadamard_test_circuit(u, v)),
                     ("swap_test", swap_test_circuit(u, v))]:
        tqc = transpile(qc, backend, optimization_level=3) # LOCAL compile, free
        ops = tqc.count_ops()
        twoq = sum(c for g, c in ops.items() if g in ("cx", "cz", "ecr"))
        print(f"\n{name} transpiled to {backend.name}:")
        print(f" qubits used : {tqc.num_qubits}")
        print(f" depth : {tqc.depth()}")
        print(f" 2-qubit gates : {twoq} (the main error + cost driver)")
        print(f" total ops : {dict(ops)}")
    print(f"\nshots per job : {shots}")
    print("cost note: 1 job per pair. A single pair = 1 tiny job (seconds of QPU).")
    print("Nothing was submitted. To actually run: --backend ibm --run --pairs 1")

if __name__ == "__main__":
    ap = argparse.ArgumentParser()
    ap.add_argument("--backend", default="aer",
                    choices=["aer", "aer_noisy", "ibm", "braket"])
    ap.add_argument("--shots", type=int, default=4096)
    ap.add_argument("--seed", type=int, default=7)
    ap.add_argument("--check", action="store_true",
                    help="IBM preflight only: auth + device + transpile, NO job")
    ap.add_argument("--run", action="store_true",
                    help="required to actually submit an IBM/braket hardware job")
    ap.add_argument("--pairs", type=int, default=5,
                    help="number of single-pair estimates in the demo (safety cap)")
    ap.add_argument("--dim", type=int, default=4,
                    help="vector dimension (must be a power of 2). dim=2 -> 1 data "
                          "qubit -> ~2 two-qubit gates, the recommended low-depth "
                          "first hardware run; dim=4 -> ~27 on a heavy-hex device")
    args = ap.parse_args()

```

```

if args.check:
    preflight(args.shots)
elif args.backend in ("ibm", "braket") and not args.run:
    print("Refusing to submit a hardware job without --run. "
          "Use --check for a free preflight, or add --run to execute.")
else:
    # on real hardware, --pairs 1 does exactly one pair and skips the
    # multi-job kernel; the kernel only runs on simulators or when pairs>=5.
    hw = args.backend in ("ibm", "braket")
    demo(args.backend, args.shots, args.seed, n_pairs=args.pairs,
          do_kernel=(not hw) or args.pairs >= 5, dim=args.dim)

# -----
# noise-model-bound runner (added for the error-mitigation studies)
# -----
def make_aer_runner(noise_model=None, seed=None):
    """Return a run(circuit, shots) -> counts bound to a specific Aer noise model.

    Unlike get_runner("aer_noisy") (which hard-codes one gate-only model), this
    takes any NoiseModel (e.g. from src.noise_models, including readout error),
    so the mitigation code can run the estimator circuits, the readout-calibration
    preparation circuits, and the ZNE-folded circuits all under the SAME model.
    noise_model=None gives the noiseless simulator.
    """
    from qiskit_aer import AerSimulator
    from qiskit import transpile
    sim = AerSimulator(noise_model=noise_model, seed_simulator=seed)

    def run(circuit, shots):
        tqc = transpile(circuit, sim)
        return sim.run(tqc, shots=shots).result().get_counts()
    return run

```

## 4.7 src/studies.py

```

"""Quantitative studies: how the quantum cosine-similarity estimate degrades with
(1) shot count and (2) hardware noise. These produce the plots + JSON that make
the "results" section of the writeup.

Run: python -m src.studies # writes results/*.json + results/*.png
"""
from __future__ import annotations
import json
import os
import numpy as np

from .classical import cosine_similarity
from .experiment import get_runner, make_estimators

RESULTS = os.path.join(os.path.dirname(os.path.dirname(os.path.abspath(__file__))), "results")
os.makedirs(RESULTS, exist_ok=True)

def shot_scaling(shots_grid=(128, 256, 512, 1024, 2048, 4096, 8192, 16384),
                 n_pairs=40, dim=4, seed=11):
    """Mean |error| of the Hadamard-test cosine vs shots. Theory: error ~ 1/sqrt(S)."""
    rng = np.random.default_rng(seed)
    _, hada = make_estimators(get_runner("aer"))
    pairs = [(rng.normal(size=dim), rng.normal(size=dim)) for _ in range(n_pairs)]
    truth = [cosine_similarity(u, v) for u, v in pairs]
    out = []
    for S in shots_grid:
        errs = [abs(hada(u, v, S) - t) for (u, v), t in zip(pairs, truth)]
        out.append({"shots": int(S), "mean_abs_error": float(np.mean(errs)),
                    "std_abs_error": float(np.std(errs))})
    print(f"shots={S:>6} mean|err|={np.mean(errs):.4f}")
    return out

```

```

def noise_scaling(noise_grid=(0.0, 0.002, 0.005, 0.01, 0.02, 0.05),
                 n_pairs=40, dim=4, shots=8192, seed=13):
    """Mean |error| vs two-qubit depolarizing rate (fixed shots)."""
    from qiskit_aer import AerSimulator
    from qiskit_aer.noise import NoiseModel, depolarizing_error
    from qiskit import transpile
    from .hadamard_test import hadamard_test_circuit, signed_inner_from_counts

    rng = np.random.default_rng(seed)
    pairs = [(rng.normal(size=dim), rng.normal(size=dim)) for _ in range(n_pairs)]
    truth = [cosine_similarity(u, v) for u, v in pairs]
    out = []
    for p2 in noise_grid:
        nm = NoiseModel()
        if p2 > 0:
            # 1q error on single-qubit gates; 2q error on cx only (cswap decomposes
            # to cx during transpile, so the cx error covers it).
            nm.add_all_qubit_quantum_error(depolarizing_error(p2 / 10, 1), ["u", "h", "x", "sdg", "rz", "sx"])
            nm.add_all_qubit_quantum_error(depolarizing_error(p2, 2), ["cx", "cz", "ecr"])
        sim = AerSimulator(noise_model=nm if p2 > 0 else None)
        errs = []
        for (u, v), t in zip(pairs, truth):
            qc = transpile(hadamard_test_circuit(u, v), sim)
            counts = sim.run(qc, shots=shots).result().get_counts()
            errs.append(abs(signed_inner_from_counts(counts, shots) - t))
        out.append({"depol_2q": float(p2), "mean_abs_error": float(np.mean(errs))})
    print(f" depol_2q={p2:<6} mean|err|={np.mean(errs):.4f}")
    return out

def make_plots(shot_rows, noise_rows):
    import matplotlib
    matplotlib.use("Agg")
    import matplotlib.pyplot as plt

    # shot scaling (log-log, with 1/sqrt(S) reference)
    S = np.array([r["shots"] for r in shot_rows])
    e = np.array([r["mean_abs_error"] for r in shot_rows])
    fig, ax = plt.subplots(figsize=(5.2, 3.6))
    ax.loglog(S, e, "o-", label="Hadamard-test error")
    ax.loglog(S, e[0] * np.sqrt(S[0]) / np.sqrt(S), "--", color="gray",
              label=r"$\propto 1/\sqrt{S}$ (shot-noise theory)")
    ax.set_xlabel("shots S"); ax.set_ylabel("mean |cos error|")
    ax.set_title("Cosine-similarity error vs shots (noiseless)"); ax.legend()
    fig.tight_layout(); fig.savefig(os.path.join(RESULTS, "shot_scaling.png"), dpi=140)

    # noise scaling
    p = np.array([r["depol_2q"] for r in noise_rows])
    en = np.array([r["mean_abs_error"] for r in noise_rows])
    fig, ax = plt.subplots(figsize=(5.2, 3.6))
    ax.plot(p * 100, en, "s-", color="#c0606a")
    ax.set_xlabel("two-qubit depolarizing rate (%)"); ax.set_ylabel("mean |cos error|")
    ax.set_title("Cosine-similarity error vs hardware noise (8192 shots)")
    fig.tight_layout(); fig.savefig(os.path.join(RESULTS, "noise_scaling.png"), dpi=140)
    print(f" wrote {RESULTS}/shot_scaling.png + noise_scaling.png")

def main():
    print("=== shot scaling (noiseless, error ~ 1/sqrt(S)) ===")
    shot_rows = shot_scaling()
    print("=== noise scaling (depolarizing) ===")
    noise_rows = noise_scaling()
    with open(os.path.join(RESULTS, "studies.json"), "w") as f:
        json.dump({"shot_scaling": shot_rows, "noise_scaling": noise_rows}, f, indent=2)
    try:
        make_plots(shot_rows, noise_rows)
    except Exception as e:
        print(f" (plot step skipped: {e})")
    print(f" wrote {RESULTS}/studies.json")

```

```
if __name__ == "__main__":
    main()
```

## 4.8 src/qsvm.py

*"""Quantum-kernel SVM: use the quantum-estimated cosine-similarity kernel as the kernel of a support-vector machine on a real dataset, and compare accuracy to the exact classical kernel. This is the capstone that turns the primitive into a working ML pipeline (and the piece that speaks to the quantum-ML PIs)."""*

*Pipeline:*

1. Load a small real dataset (sklearn Iris), reduce to `dim` features (PCA).
2. Build the training kernel  $K_{\text{train}}$  and test kernel  $K_{\text{test}}$  by estimating cosine similarity between every pair -- once CLASSICALLY (exact) and once with the QUANTUM Hadamard-test estimator on the simulator.
3. Fit sklearn SVC(kernel='precomputed') on each and compare test accuracy.

*The point: a kernel assembled entirely from quantum overlap measurements still drives a classifier to (near) the same accuracy as the exact kernel -- with a controlled, measurable degradation from shot noise.*

*Run: python -m src.qsvm*  
*"""*

```
from __future__ import annotations
import json
import os
import numpy as np
```

```
from .classical import normalize, kernel_matrix
from .experiment import get_runner, make_estimators
```

```
RESULTS = os.path.join(os.path.dirname(os.path.abspath(__file__)), "results")
os.makedirs(RESULTS, exist_ok=True)
```

```
def _cross_kernel(A, B, estimator, shots):
    """Similarity matrix between rows of A and rows of B via a quantum estimator."""
    K = np.zeros((len(A), len(B)))
    for i in range(len(A)):
        for j in range(len(B)):
            K[i, j] = estimator(A[i], B[j], shots)
    return K
```

```
def run(dim=4, n_per_class=20, shots=4096, seed=0):
    from sklearn.datasets import load_iris
    from sklearn.decomposition import PCA
    from sklearn.model_selection import train_test_split
    from sklearn.svm import SVC
    from sklearn.metrics import accuracy_score

    # 1) data -> dim features, two classes for a clean binary task
    X, y = load_iris(return_X_y=True)
    mask = y < 2 # classes 0 and 1
    X, y = X[mask], y[mask]
    X = PCA(n_components=dim, random_state=seed).fit_transform(X)
    X = normalize(X) # unit vectors -> cos = inner product
    Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.4, random_state=seed, stratify=y)

    # 2) exact (classical) kernels
    Ktr_c = kernel_matrix(Xtr)
    Kte_c = normalize(Xte) @ normalize(Xtr).T

    # 3) quantum kernels (Hadamard test on the simulator)
    _, hada = make_estimators(get_runner("aer"))
    Ktr_q = _sym_kernel(Xtr, hada, shots)
    Kte_q = _cross_kernel(Xte, Xtr, hada, shots)
```



```

def fit_eval(Ktr, Kte):
    clf = SVC(kernel="precomputed").fit(Ktr, ytr)
    return float(accuracy_score(yte, clf.predict(Kte)))

acc_c = fit_eval(Ktr_c, Kte_c)
acc_q = fit_eval(Ktr_q, Kte_q)
kernel_rmse = float(np.sqrt(np.mean((Ktr_q - Ktr_c) ** 2)))
out = {"dim": dim, "shots": shots, "n_train": len(Xtr), "n_test": len(Xte),
      "accuracy_classical_kernel": acc_c, "accuracy_quantum_kernel": acc_q,
      "train_kernel_rmse": kernel_rmse}
print("quantum-kernel SVM vs classical-kernel SVM:")
print(f" classical-kernel test accuracy: {acc_c:.3f}")
print(f" quantum-kernel test accuracy: {acc_q:.3f}")
print(f" train-kernel RMSE (q vs exact): {kernel_rmse:.4f}")
with open(os.path.join(RESULTS, "qsvm.json"), "w") as f:
    json.dump(out, f, indent=2)
print(f"wrote {RESULTS}/qsvm.json")
return out

def _sym_kernel(X, estimator, shots):
    m = len(X)
    K = np.eye(m)
    for i in range(m):
        for j in range(i + 1, m):
            s = estimator(X[i], X[j], shots)
            K[i, j] = K[j, i] = s
    return K

if __name__ == "__main__":
    run()

```

## 4.9 src/angle\_sweep.py

```

"""Signed-cosine ANGLE SWEEP --the recommended first real-hardware experiment.

Instead of one lucky pair, sweep a single pair of 2-D vectors across a set of angles
so the run traces the estimator across the FULL signed range (cos from +1 down toward
-1). This turns one hardware session into a "measured vs. true cosine" line --the
strongest single-figure evidence that the primitive works on a device, and it exercises
the negative-cosine half that motivates using the Hadamard test at all.

WHY 2-D (dim=2, 1 data qubit)
The 4-D Hadamard-test circuit routes to ~27 two-qubit gates on a heavy-hex device;
the 2-D circuit needs a single controlled rotation -> ~2 two-qubit gates. That 13x
reduction is the difference between a washed-out and a clean first result. (See the
--dim flag rationale in experiment.py.)

TWO STRUCTURAL FACTS (found while validating this design)
1. cos = -1 is EXCLUDED, not a bug. For antiparallel vectors the encoded states differ
   only by a global phase ( $|\psi_v\rangle = -|\psi_u\rangle$ ), which amplitude encoding correctly
   discards and the single-register Hadamard test cannot observe -- so cos=-1 reads back
   as +1. The sweep therefore stops at ~160 deg (cos ~ -0.94). If you ever see +1 where
   you expect -1 on hardware, this is why, not noise.
2. Neither vector is a basis state. A pure basis state encodes with zero real rotations
   -- unrepresentative of what the device runs. We offset both vectors by OFFSET rad so
   every swept circuit carries a genuine rotation.

PREDICTION BAND
Each point's predicted error is read from results/studies.json. That curve is the dim=4
UPPER BOUND (~27 gates); the actual dim=2 circuit has ~2, so studies.json OVERPREDICTS
by design. We label the band "dim=4 upper bound" rather than silently rescaling -- the
honest read for the first hardware run is: expect errors at or below the band, and the
gap is the low-depth win made visible.

SAFETY
Gated exactly like experiment.py: `--check` runs a free IBM preflight (transpile +
predict, NO job); a real submission requires `--backend ibm --run`. Simulators are the

```

default and submit nothing.

Run:

```
python -m src.angle_sweep # noiseless + noisy simulator sweep + plot
python -m src.angle_sweep --backend aer_noisy
python -m src.angle_sweep --backend ibm --check # free preflight, no job
python -m src.angle_sweep --backend ibm --run # submit the sweep (6 tiny jobs)
"""
from __future__ import annotations
import argparse
import json
import os
import numpy as np

from .classical import cosine_similarity
from .experiment import get_runner, make_estimators

ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
RESULTS = os.path.join(ROOT, "results")

OFFSET = 0.4 # rad: keep both vectors off the basis states
MAX_DEG = 160.0 # stop short of 180 (cos=-1 is a global-phase degeneracy)

def sweep_pairs(n=6):
    """n 2-D unit-vector pairs whose true cosine walks from +1 toward ~-0.94.
    Both vectors carry the OFFSET so neither is a computational basis state."""
    thetas = np.linspace(0.0, np.deg2rad(MAX_DEG), n)
    pairs = []
    for dtheta in thetas:
        u = np.array([np.cos(OFFSET), np.sin(OFFSET)])
        v = np.array([np.cos(OFFSET + dtheta), np.sin(OFFSET + dtheta)])
        pairs.append((u, v))
    return pairs

def predicted_band(depol_2q=0.02):
    """Predicted mean |cos err| at a given 2q depolarizing rate, read from the
    noise sweep in studies.json. This is the dim=4 upper bound (conservative)."""
    try:
        with open(os.path.join(RESULTS, "studies.json")) as f:
            rows = json.load(f)["noise_scaling"]
            # nearest tabulated rate
            row = min(rows, key=lambda r: abs(r["depol_2q"] - depol_2q))
            return row["mean_abs_error"], row["depol_2q"]
    except Exception:
        return None, None

def run_sweep(backend="aer", shots=8192, n=6):
    runner = get_runner(backend)
    _, hada = make_estimators(runner)
    band, rate = predicted_band()
    print(f"\n=== signed-cosine angle sweep (backend={backend}, shots={shots}, "
          f"pairs={n}, dim=2) ===")
    if band is not None:
        print(f"predicted |err| band (dim=4 UPPER bound @ {rate*100:.0f}% 2q noise): "
              f"<= {band:.3f}")
    print(f"{'true cos':>10}{'estimate':>11}{'|err|':>9}")
    rows = []
    for u, v in sweep_pairs(n):
        true = cosine_similarity(u, v)
        est = hada(u, v, shots)
        rows.append((true, est, abs(est - true)))
        print(f"{'true:>10.3f'}{'est:>11.3f'}{'abs(est - true):>9.3f}")
    return rows, band

def make_plot(rows_noiseless, rows_noisy, band):
    import matplotlib
    matplotlib.use("Agg")
```

```

import matplotlib.pyplot as plt
t = [r[0] for r in rows_noiseless]
fig, ax = plt.subplots(figsize=(5.6, 4.0))
ax.plot([-1, 1], [-1, 1], "--", color="gray", label="ideal (measured = true)")
ax.plot(t, [r[1] for r in rows_noiseless], "o-", color="#5b8cff",
        label="noiseless sim")
if rows_noisy:
    ax.plot(t, [r[1] for r in rows_noisy], "s-", color="#c0606a",
            label="sim hardware (2% 2q)")
if band:
    ax.fill_between([-1, 1], [-1 - band, 1 - band], [-1 + band, 1 + band],
                    color="#c0606a", alpha=0.10,
                    label=f"predicted band (dim=4 upper, +/-{band:.02f})")
ax.set_xlabel("true cosine similarity"); ax.set_ylabel("estimated cosine")
ax.set_title("Signed-cosine sweep on the Hadamard test (dim=2)")
ax.legend(fontsize=8); fig.tight_layout()
out = os.path.join(RESULTS, "angle_sweep.png")
fig.savefig(out, dpi=140)
print(f"wrote {out}")

def preflight(shots=8192, n=6):
    """QPU-FREE preflight: auth, pick device, transpile the 2-D sweep circuits,
    report 2-qubit-gate cost + the predicted band. Submits NO job."""
    from qiskit import transpile
    from qiskit_ibm_runtime import QiskitRuntimeService
    from .hadamard_test import hadamard_test_circuit
    print("=== IBM preflight for angle sweep (NO job submitted) ===")
    service = QiskitRuntimeService()
    backend = service.least_busy(operational=True, simulator=False)
    print(f"device: {backend.name} ({backend.num_qubits}q, "
          f"{getattr(backend.status(), 'pending_jobs', '?')} queued)")
    twoq_total = 0
    for u, v in sweep_pairs(n):
        tqc = transpile(hadamard_test_circuit(u, v), backend, optimization_level=3)
        twoq = sum(c for g, c in tqc.count_ops().items() if g in ("cz", "cx", "ecr"))
        twoq_total += twoq
    band, rate = predicted_band()
    print(f"circuits: {n} | avg 2-qubit gates/circuit: {twoq_total / n:.1f} "
          f"(vs ~27 for dim=4)")
    print(f"predicted |err| <= {band:.3f} (dim=4 upper bound @ {rate*100:.0f}% 2q)")
    print(f"shots/job {shots} x {n} jobs = seconds of QPU. Nothing submitted.")

def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--backend", default="aer",
                    choices=["aer", "aer_noisy", "ibm", "braket"])
    ap.add_argument("--shots", type=int, default=8192)
    ap.add_argument("--pairs", type=int, default=6)
    ap.add_argument("--check", action="store_true",
                    help="IBM preflight only (transpile + predict), NO job")
    ap.add_argument("--run", action="store_true",
                    help="required to actually submit an ibm/braket job")
    ap.add_argument("--no-plot", action="store_true")
    args = ap.parse_args()

    if args.check:
        preflight(args.shots, args.pairs)
        return
    if args.backend in ("ibm", "braket") and not args.run:
        print("Refusing to submit a hardware job without --run. "
              "Use --check for a free preflight.")
        return

    rows, band = run_sweep(args.backend, args.shots, args.pairs)
    rows_noisy = None
    if args.backend == "aer": # add a simulated-hardware comparison line
        rows_noisy, _ = run_sweep("aer_noisy", args.shots, args.pairs)
    if not args.no_plot:
        try:

```

```

        make_plot(rows, rows_noisy, band)
    except Exception as e:
        print(f"(plot skipped: {e})")

if __name__ == "__main__":
    main()

```

## 4.10 src/mitigation.py

```

"""Error mitigation for the overlap estimators: readout calibration + ZNE.

Two techniques, both classical post-processing / circuit-level tricks that need
no change to the physics of the swap or Hadamard test:

1. READOUT-ERROR CALIBRATION (this file, part A)
   Measure the device's readout confusion matrix by preparing known bit
   strings, then invert it to undo the measurement bias in the raw counts.

2. ZERO-NOISE EXTRAPOLATION (ZNE) (this file, part B)
   Deliberately amplify the GATE noise by folding the circuit
   (G -> G G^dag G, exact identity in the noiseless limit), measure the
   observable at several noise scale factors, and extrapolate back to the
   zero-noise limit.

Both are standard NISQ-era methods (Kandala et al. 2019 for ZNE; the calibration
-matrix approach for readout goes back to the earliest IBM devices). We
implement them by hand -- a few dozen lines each -- rather than pulling a
library, so the math is visible and the dependency stays numpy + qiskit.

=====
PART A -- READOUT-ERROR CALIBRATION
=====

THE MODEL
The device does not report the true measured bitstring t; it reports an observed
bitstring o drawn from a fixed conditional distribution P(o | t). Collect those
into a 2^k x 2^k column-stochastic matrix A (k = number of measured qubits):

    A[o, t] = P(observe o | true state was t), sum_o A[o, t] = 1.

If p_true is the (unknown) probability vector we WANT and p_obs is what the
device gives, then

    p_obs = A p_true.

CALIBRATION
We learn A directly: for each basis state t in {0,1}^k, run a circuit that
prepares |t> and measures immediately; the normalized histogram of outcomes is
column t of A. For the swap / Hadamard tests only ONE qubit is measured (the
ancilla), so A is 2x2 and needs just two calibration circuits (|0>, |1>).

INVERSION
Given A and p_obs we recover p_true. The naive p_true = A^{-1} p_obs can produce
negative or >1 "probabilities" because of shot noise. We instead solve the
constrained least-squares problem

    min_x || A x - p_obs ||^2 s.t. x_i >= 0, sum_i x_i = 1,

which always returns a valid probability vector. For the common 2x2 case this is
cheap; for k>1 we use scipy's SLSQP. The corrected P(0) then feeds the SAME
2 P(0) - 1 conversion as before.
"""
from __future__ import annotations
import itertools
import numpy as np

# -----

```

```

# A.1 build the calibration (confusion) matrix
# -----
def calibration_matrix(runner, measured_qubits, total_qubits, shots=8192):
    """Measure the readout confusion matrix  $A[o, t] = P(\text{observe } o \mid \text{prepared } t)$ .

    Parameters
    -----
    runner : run(circuit, shots) -> counts, bound to the noisy backend.
    measured_qubits : list of qubit indices whose readout we calibrate (e.g.
        [0] for the single ancilla of the swap/Hadamard test).
    total_qubits : width of the register the circuit lives on (so the prep
        circuit matches the estimator circuit's layout).
    shots : shots per calibration circuit.

    Returns a  $(2^k, 2^k)$  column-stochastic numpy array,  $k = \text{len}(\text{measured\_qubits})$ .
    Bitstring index convention matches _counts_to_vec below (qubit order =
    the order of measured_qubits, first listed = most significant bit).
    """
    from qiskit import QuantumCircuit
    k = len(measured_qubits)
    dim = 2 ** k
    A = np.zeros((dim, dim))
    for t in range(dim):
        bits = [(t >> (k - 1 - i)) & 1 for i in range(k)] # MSB-first
        qc = QuantumCircuit(total_qubits, k)
        for i, (q, b) in enumerate(zip(measured_qubits, bits)):
            if b:
                qc.x(q)
        qc.measure(measured_qubits, list(range(k)))
        counts = runner(qc, shots)
        A[:, t] = _counts_to_vec(counts, k, shots)
    return A

def _counts_to_vec(counts, k, shots):
    """Normalized outcome-probability vector of length  $2^k$  from a counts dict.

    Qiskit returns the classical register as a string; bit 0 of that string is
    the LAST classical bit. We measured measured_qubits -> classical 0..k-1, so
    the reported string has classical bit (k-1) first. We read it MSB-first
    (classical bit 0 = most significant) to match calibration_matrix's index t.
    """
    vec = np.zeros(2 ** k)
    for bitstr, n in counts.items():
        s = bitstr.replace(" ", "")
        s = s[-k:].zfill(k) # keep the k measured bits
        idx = int(s, 2)
        vec[idx] += n
    total = vec.sum()
    return vec / total if total > 0 else vec

# -----
# A.2 invert: corrected probability vector from observed counts
# -----
def correct_counts(A, counts, shots):
    """Return a corrected probability vector  $p_{\text{true}}$  solving the constrained LSQ
     $\min ||A x - p_{\text{obs}}||^2$  s.t.  $x \geq 0$ ,  $\sum x = 1$ .
    """
    k = int(round(np.log2(A.shape[0])))
    p_obs = _counts_to_vec(counts, k, shots)
    return _constrained_inverse(A, p_obs)

def _constrained_inverse(A, p_obs):
    """Solve  $\min ||A x - p_{\text{obs}}||^2$  s.t.  $x \geq 0$ ,  $\sum x = 1$ . Returns  $x$  (prob vector)."""
    dim = A.shape[1]
    # analytic unconstrained solution first; if it is already a valid simplex
    # point we can skip the optimizer (the common, low-noise case).
    try:
        x0 = np.linalg.solve(A, p_obs)

```

```

except np.linalg.LinAlgError:
    x0, *_ = np.linalg.lstsq(A, p_obs, rcond=None)
    if np.all(x0 >= -1e-9) and abs(x0.sum() - 1) < 1e-9:
        return np.clip(x0, 0, None) / np.clip(x0, 0, None).sum()

from scipy.optimize import minimize
cons = ({'type': 'eq', 'fun': lambda x: x.sum() - 1.0},)
bounds = [(0.0, 1.0)] * dim
guess = np.clip(x0, 0, 1)
guess = guess / guess.sum() if guess.sum() > 0 else np.ones(dim) / dim
res = minimize(lambda x: float(np.sum((A @ x - p_obs) ** 2)),
               guess, method='SLSQP', bounds=bounds, constraints=cons,
               options={'maxiter': 200, 'ftol': 1e-12})
x = np.clip(res.x, 0, None)
return x / x.sum() if x.sum() > 0 else x

# -----
# A.3 readout-mitigated single-ancilla P(0) (swap / Hadamard test)
# -----
def mitigated_p0_single_ancilla(runner, circuit, A, shots):
    """Run `circuit`, correct its ancilla readout with A, return corrected P(0).

    The swap and Hadamard tests both measure exactly one qubit (the ancilla) into
    a 1-bit classical register, so A is 2x2 and the corrected P(0) is x[0].
    """
    counts = runner(circuit, shots)
    x = correct_counts(A, counts, shots)
    return float(x[0])

# =====
# PART B -- ZERO-NOISE EXTRAPOLATION (ZNE)
# =====
#
# IDEA
# The observable we read (here P(0), or the cosine derived from it) degrades as
# gate noise grows. If we could DIAL noise up we would see a trend and could
# extrapolate back to zero. We dial it up without touching the hardware by
# UNITARY FOLDING: replace a (sub)circuit G by
#
# G -> G (G^dag G)^m, lambda = 2m + 1 in {1, 3, 5, ...}
#
# which equals G exactly (G^dag G = I) but runs 2m extra noisy copies, so the
# accumulated gate error is ~lambda times larger. Measuring the observable
# E(lambda) at lambda = 1, 3, 5 and fitting E(lambda) -> lambda=0 estimates the
# zero-noise value E(0).
#
# We fold the WHOLE circuit up to (but not including) the final measurement --
# "global folding". A local variant that folds only the 2-qubit gates is also
# provided; it amplifies the dominant error source with less single-qubit
# overhead. Readout error is NOT amplified by folding (the single final measure
# is untouched), which is the intended separation of concerns: readout
# calibration handles readout, ZNE handles gate noise. Stacking both is the
# "+both" arm of the error budget.
#
# EXTRAPOLATION FITS
# - linear: E(lambda) = a + b lambda ; report a. Robust, few points.
# - richardson: exact polynomial through all points, evaluated at 0
#   (equivalent to the highest-order fit the points allow).
# - exponential: E(lambda) = E_inf + (E0 - E_inf) exp(-k lambda); the physically
#   motivated form for depolarizing decay toward the mixed state.
# We default to linear (fewest assumptions) and report the others for comparison.

def fold_global(circuit, scale):
    """Global unitary folding to an ODD integer scale factor (1, 3, 5, ...).

    Returns a new circuit equal to the original up to the final measurements but
    with gate count ~scale x. scale=1 returns the circuit unchanged.
    """
    if scale == 1:

```

```

    return circuit.copy()
if scale % 2 == 0 or scale < 1:
    raise ValueError("global fold scale must be a positive odd integer")
m = (scale - 1) // 2

from qiskit import QuantumCircuit
# split off trailing measurements so we fold only the unitary part
unitary = circuit.copy_empty_like()
measures = []
for instr in circuit.data:
    if instr.operation.name in ("measure", "barrier"):
        measures.append(instr)
    else:
        unitary.append(instr.operation, instr.qubits, instr.clbits)

folded = circuit.copy_empty_like()
folded.compose(unitary, inplace=True)
inv = unitary.inverse()
for _ in range(m):
    folded.compose(inv, inplace=True)
    folded.compose(unitary, inplace=True)
for instr in measures:
    folded.append(instr.operation, instr.qubits, instr.clbits)
return folded

def fold_local_2q(circuit, scale):
    """Local folding: replace each 2-qubit gate g by g (g† g)m.

    Amplifies the dominant (2-qubit) gate error while leaving 1-qubit gates and
    the final measurement alone. scale must be an odd integer; scale=1 is a
    no-op copy.

    IMPORTANT: this folds gates by NAME, so the circuit must already be
    decomposed to a basis whose entangling gates are in `two_q` below (e.g.
    transpile(circuit, basis_gates=["u","cx"]) first). On a high-level circuit
    whose entanglers are still wrapped in a custom gate (like the Hadamard
    test's controlled-U), it finds nothing to fold and returns the circuit
    unchanged. The error-budget study uses GLOBAL folding, which has no such
    requirement.
    """
    if scale == 1:
        return circuit.copy()
    if scale % 2 == 0 or scale < 1:
        raise ValueError("local fold scale must be a positive odd integer")
    m = (scale - 1) // 2

    two_q = {"cx", "cz", "ecr", "cswap", "ccx"}
    folded = circuit.copy_empty_like()
    for instr in circuit.data:
        op = instr.operation
        folded.append(op, instr.qubits, instr.clbits)
        if op.name in two_q:
            for _ in range(m):
                folded.append(op.inverse(), instr.qubits, instr.clbits)
                folded.append(op, instr.qubits, instr.clbits)
    return folded

def zne_estimate(runner, circuit, observable_fn, shots=8192, scales=(1, 3, 5),
                 fold="global", fit="richardson", clamp=(-1.0, 1.0)):
    """Zero-noise-extrapolate a scalar observable of `circuit`.

    Parameters
    -----
    runner : run(circuit, shots) -> counts, bound to the noisy backend.
    observable_fn : callable(counts, shots) -> float, the observable to
        extrapolate (e.g. lambda c, s: 2*p0(c, s)-1 for the cosine).
    shots : shots per folded circuit.
    scales : odd integer noise scale factors to sample.
    fold : "global" or "local2q".

```

```

fit : "linear" | "richardson" | "exponential".

`clamp` bounds the extrapolated value to a physical range (default the
cosine range [-1, 1]); pass clamp=None to disable. Clamping is essential:
a degree-(k-1) polynomial through a few noisy points can extrapolate to a
non-physical value, and an un-clamped outlier destroys a mean-error summary.

Returns dict {scales, values, extrapolated, extrapolated_raw, fit}.
"""
folder = fold_global if fold == "global" else fold_local_2q
values = []
for sc in scales:
    fc = folder(circuit, sc)
    counts = runner(fc, shots)
    values.append(observable_fn(counts, shots))
scales = np.asarray(scales, dtype=float)
values = np.asarray(values, dtype=float)
ext_raw = _extrapolate(scales, values, fit)
ext = ext_raw if clamp is None else float(np.clip(ext_raw, clamp[0], clamp[1]))
return {"scales": scales.tolist(), "values": values.tolist(),
        "extrapolated": float(ext), "extrapolated_raw": float(ext_raw),
        "fit": fit}

def _extrapolate(scales, values, fit):
    """Extrapolate values(scales) to scale=0 with the chosen fit."""
    if fit == "linear":
        b, a = np.polyfit(scales, values, 1) # values = b*scale + a
        return a
    if fit == "richardson":
        # exact interpolating polynomial of degree len-1, evaluated at 0
        coeffs = np.polyfit(scales, values, len(scales) - 1)
        return np.polyval(coeffs, 0.0)
    if fit == "exponential":
        #  $E(l) = E_{\text{inf}} + (E_0 - E_{\text{inf}}) \exp(-k l)$ ; depolarizing decays toward the
        # maximally mixed value  $E_{\text{inf}}$ . Fit with a bounded curve_fit, fall back
        # to linear if it fails to converge.
        from scipy.optimize import curve_fit
        def model(l, E_inf, amp, k):
            return E_inf + amp * np.exp(-k * l)
        try:
            import warnings
            p0 = (values[-1], values[0] - values[-1], 0.3)
            with warnings.catch_warnings():
                warnings.simplefilter("ignore")
                popt, _ = curve_fit(model, scales, values, p0=p0, maxfev=10000)
            return model(0.0, *popt)
        except Exception:
            b, a = np.polyfit(scales, values, 1)
            return a
    raise ValueError(f"unknown fit {fit!r}")

```

## 4.11 src/mitigation\_study.py

```

"""Error-budget study: raw vs readout-corrected vs readout+ZNE on a realistic noise
model (two-qubit depolarizing + readout error). Produces the before/after numbers and
a figure for the writeup. Uses the mitigation primitives in `mitigation.py`.

Run: python -m src.mitigation_study
"""
from __future__ import annotations
import json
import os
import numpy as np

from .classical import cosine_similarity
from .encode import num_qubits
from .hadamard_test import hadamard_test_circuit
from .mitigation import calibration_matrix, correct_counts, zne_estimate

```



```

ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
RESULTS = os.path.join(ROOT, "results")

def noisy_runner(depol_2q=0.02, readout=0.02):
    """AerSimulator with two-qubit depolarizing + symmetric readout error –
    the two dominant channels on a superconducting device."""
    from qiskit_aer import AerSimulator
    from qiskit_aer.noise import NoiseModel, depolarizing_error, ReadoutError
    from qiskit import transpile
    nm = NoiseModel()
    nm.add_all_qubit_quantum_error(depolarizing_error(depol_2q / 10, 1),
                                   ["u", "h", "x", "sdg", "rz", "sx"])
    nm.add_all_qubit_quantum_error(depolarizing_error(depol_2q, 2), ["cx", "cz", "ecr"])
    nm.add_all_qubit_readout_error(ReadoutError([[1 - readout, readout],
                                                  [readout, 1 - readout]]))
    sim = AerSimulator(noise_model=nm)

    def run(circuit, shots):
        tqc = transpile(circuit, sim)
        return sim.run(tqc, shots=shots).result().get_counts()
    return run

def _p0(counts, shots):
    return counts.get("0", 0) / shots

def run_study(n_pairs=30, dim=4, shots=8192, scales=(1, 3, 5), seed=17):
    runner = noisy_runner()
    n = max(1, dim.bit_length() - 1) # data qubits
    total_q = n + 1 # + ancilla (measured = qubit 0)
    A = calibration_matrix(runner, [0], total_q, shots) # readout assignment matrix

    def cos_raw(counts, shots): # signed cosine, no correction
        return 2 * _p0(counts, shots) - 1

    def cos_readout(counts, shots): # readout-corrected signed cosine
        return 2 * correct_counts(A, counts, shots)[0] - 1

    rng = np.random.default_rng(seed)
    pairs = [(rng.normal(size=dim), rng.normal(size=dim)) for _ in range(n_pairs)]
    e_raw, e_ro, e_zne = [], [], []
    for u, v in pairs:
        true = cosine_similarity(u, v)
        circ = hadamard_test_circuit(u, v)
        raw = cos_raw(runner(circ, shots), shots) # no mitigation
        ro = cos_readout(runner(circ, shots), shots) # readout only
        zne = zne_estimate(runner, circ, cos_readout, shots, scales)["extrapolated"]
        e_raw.append(abs(np.clip(raw, -1, 1) - true))
        e_ro.append(abs(np.clip(ro, -1, 1) - true))
        e_zne.append(abs(np.clip(zne, -1, 1) - true))

    mae = {"raw": round(float(np.mean(e_raw)), 4),
           "readout_corrected": round(float(np.mean(e_ro)), 4),
           "readout_plus_zne": round(float(np.mean(e_zne)), 4)}
    out = {"n_pairs": n_pairs, "dim": dim, "shots": shots, "scales": list(scales),
           "depol_2q": 0.02, "readout": 0.02, "mean_abs_error": mae,
           "assignment_matrix": A.round(4).tolist()}
    print("=== error budget (mean |cos error|, 30 pairs, 2% 2q depol + 2% readout) ===")
    for k, v in mae.items():
        print(f" {k:20} {v:.4f}")
    red = 100 * (1 - mae["readout_plus_zne"] / mae["raw"]) if mae["raw"] else 0
    print(f" -> mitigation cuts mean error by {red:.0f}%")
    with open(os.path.join(RESULTS, "mitigation.json"), "w") as f:
        json.dump(out, f, indent=2)
    print(f"wrote {RESULTS}/mitigation.json")
    return out

```

```

def make_plot(out):
    import matplotlib
    matplotlib.use("Agg")
    import matplotlib.pyplot as plt
    mae = out["mean_abs_error"]
    labels = ["raw", "readout\ncorrected", "readout\n+ ZNE"]
    vals = [mae["raw"], mae["readout_corrected"], mae["readout_plus_zne"]]
    fig, ax = plt.subplots(figsize=(4.8, 3.6))
    bars = ax.bar(labels, vals, color=["#c0606a", "#e0b450", "#46c08a"])
    for b, v in zip(bars, vals):
        ax.text(b.get_x() + b.get_width() / 2, v + 0.002, f"{v:.3f}",
                ha="center", fontsize=9)
    ax.set_ylabel("mean |cosine error|")
    ax.set_title("Error mitigation on the Hadamard test\n(2% 2q depol + 2% readout, 30 pairs)")
    fig.tight_layout()
    out_png = os.path.join(RESULTS, "mitigation.png")
    fig.savefig(out_png, dpi=140)
    print(f"wrote {out_png}")

if __name__ == "__main__":
    out = run_study()
    try:
        make_plot(out)
    except Exception as e:
        print(f"(plot skipped: {e})")

```

## 4.12 tests/test\_estimators.py

```

"""Correctness tests: prove the quantum estimators reproduce the classical math.

These run on the noiseless simulator with many shots, so they are tight. They are
the "does the physics actually compute cosine similarity?" proof -- the thing a
skeptical PI (or you) should check first.

"""

import numpy as np
import sys, os
sys.path.insert(0, os.path.dirname(os.path.dirname(os.path.abspath(__file__))))

from src.encode import normalized_state, num_qubits
from src.classical import cosine_similarity, kernel_matrix
from src.experiment import get_runner, make_estimators
from src.kernel import quantum_kernel_matrix, kernel_error

def test_encoding_is_unit_and_matches():
    x = np.array([3.0, 4.0]) # ||x|| = 5
    a = normalized_state(x)
    assert abs(np.linalg.norm(a) - 1.0) < 1e-12
    assert np.allclose(a, [0.6, 0.8])
    assert num_qubits(np.zeros(1024)) == 10

def test_swap_test_matches_abs_cosine():
    # The swap test measures |<u|v>|^2 LINEARLY in the ancilla probability; the
    # sqrt that recovers |cos| amplifies shot noise near 0, so we check the
    # squared quantity (what the device actually estimates) for a tight bound,
    # and |cos| itself with a looser tolerance.
    runner = get_runner("aer")
    swap_est, _ = make_estimators(runner)
    rng = np.random.default_rng(1)
    for _ in range(6):
        u, v = rng.normal(size=4), rng.normal(size=4)
        approx = swap_est(u, v, shots=50000) # |cos| estimate
        true = abs(cosine_similarity(u, v))
        # Assert the LINEAR measured quantity (squared overlap); the sqrt that
        # recovers |cos| is ill-conditioned near 0, so we bound |cos| only away
        # from zero where the sqrt is well-behaved.
        assert abs(approx ** 2 - true ** 2) < 0.03 # squared overlap: tight, always

```

```

    if true > 0.2:
        assert abs(approx - true) < 0.05 # |cos|: only where sqrt is stable

def test_hadamard_matches_signed_cosine():
    runner = get_runner("aer")
    _, hada_est = make_estimators(runner)
    rng = np.random.default_rng(2)
    for _ in range(6):
        u, v = rng.normal(size=4), rng.normal(size=4)
        approx = hada_est(u, v, shots=20000)
        assert abs(approx - cosine_similarity(u, v)) < 0.03

def test_hadamard_complex_re_and_im():
    """Complex vectors: Re branch recovers Re<u|v>, Im branch recovers Im<u|v>.
    This exercises the imag=True path (previously unreachable when the encoder
    cast complex input to float) and confirms both real and imaginary overlap."""
    from src.hadamard_test import hadamard_test_circuit, signed_inner_from_counts
    runner = get_runner("aer")

    def nrm(x):
        x = np.asarray(x, complex)
        return x / np.linalg.norm(x)

    rng = np.random.default_rng(5)
    for _ in range(4):
        u = rng.normal(size=4) + 1j * rng.normal(size=4)
        v = rng.normal(size=4) + 1j * rng.normal(size=4)
        ref = np.vdot(nrm(u), nrm(v)) # <u|v> = sum conj(u_i) v_i (normalized)
        re = signed_inner_from_counts(runner(hadamard_test_circuit(u, v, imag=False), 40000), 40000)
        im = signed_inner_from_counts(runner(hadamard_test_circuit(u, v, imag=True), 40000), 40000)
        assert abs(re - ref.real) < 0.03
        assert abs(im - ref.imag) < 0.03

def test_mitigation_primitives():
    """Readout calibration on a noiseless runner returns the identity assignment
    matrix, and global unitary folding preserves the logical result at every noise
    scale (U (U^dag U)^k == U)."""
    from src.mitigation import calibration_matrix, fold_global
    from src.hadamard_test import hadamard_test_circuit, signed_inner_from_counts
    runner = get_runner("aer")
    A = calibration_matrix(runner, [0], 1, shots=20000)
    assert np.allclose(A, np.eye(2), atol=0.02) # no readout error -> identity
    rng = np.random.default_rng(0)
    u, v = rng.normal(size=4), rng.normal(size=4)
    true = cosine_similarity(u, v)
    for s in (1, 3, 5): # folding is logically inert
        c = runner(fold_global(hadamard_test_circuit(u, v), s), 20000)
        assert abs(signed_inner_from_counts(c, 20000) - true) < 0.03

def test_kernel_matrix_close():
    runner = get_runner("aer")
    _, hada_est = make_estimators(runner)
    rng = np.random.default_rng(3)
    X = rng.normal(size=(5, 4))
    K_q = quantum_kernel_matrix(X, hada_est, shots=20000)
    err = kernel_error(K_q, kernel_matrix(X))
    assert err["max_abs_error"] < 0.05
    assert err["diag_max_dev"] < 1e-9 # diagonal is exact by construction

if __name__ == "__main__":
    test_encoding_is_unit_and_matches()
    test_swap_test_matches_abs_cosine()
    test_hadamard_matches_signed_cosine()
    test_hadamard_complex_re_and_im()
    test_kernel_matrix_close()
    test_mitigation_primitives()

```

```
print("ALL TESTS PASSED")
```